

VIT®

Vellore Institute of Technology

(Deemed to be University under section 3 of UGC Act, 1956)

Project report

Continuum

Intent-verified recovery for
timed academic assessments

Submitted in partial fulfilment of the requirements for

Cloud Architecture Design

BCSE355L

Submitted by

Ayush Pandey

Registration Number: 24BIT0394

Under the guidance of

Venketesh P

Professor Grade 2, SCORE

Fall Semester 2026–27

Vellore Institute of Technology, Vellore

Acknowledgement

I express my sincere gratitude to **Venketesh P, Professor Grade 2, SCORE**, for guidance in the course **Cloud Architecture Design (BCSE355L)** and for the opportunity to apply cloud architecture principles to a working academic assessment platform.

I thank Vellore Institute of Technology for providing the academic setting for this project. The study of availability, fault isolation, distributed data correctness, cloud security, and operational cost has helped connect architectural concepts with implementation decisions.

The development of Continuum has reinforced the importance of testing the operation that users depend on. A server restart alone does not resolve the uncertainty of a student's final submission; recovery must also preserve the attempt, prevent conflicting writes, and return a verifiable receipt.

Ayush Pandey
24BIT0394

Abstract

Timed online assessments create a demanding recovery problem: students must be able to submit answers before an authoritative deadline, and a lost network response must not create uncertainty about a completed submission. Continuum is an implemented assessment platform that combines application recovery with transactional writer fencing and durable submission receipts.

The cloud architecture places two Go application instances in separate Availability Zones within the selected AWS Region, Sydney (`ap-southeast-2`). A React frontend is delivered through Amazon CloudFront and private Amazon S3 storage. An Amazon API Gateway HTTP API invokes a router Lambda that reads an authoritative DynamoDB control record. A scheduled monitor checks process liveness and application readiness separately. Following repeated active-specific failures, an AWS Step Functions workflow acquires an incident lease, revalidates the standby, advances the active writer epoch, verifies the public route, and reconciles pending audit work.

Final submissions bind a persistent request identifier to an immutable answer snapshot. The submission, receipt, attempt completion, and outbox entry are committed together with a condition on the writer's backend and epoch. An exact retry can retrieve an existing receipt after the assessment closes, while a changed payload under the same identifier is rejected. Local mode exercises these contracts using separate HTTP servers and shared memory; cloud mode uses separate EC2 instances and a regional DynamoDB table.

A recorded AWS rehearsal on 10 October 2026 recovered from an actual active-instance stop. The external observation interval was 50.472 seconds, while the controller's check-cycle-to-public-verification interval was 23.713 seconds. Receipt replay preserved the request identifier and commit timestamp; independent inspection found one canonical submission, one receipt, one processed outbox record, and one audit summary. These results establish a bounded prototype outcome rather than a service-level guarantee. Complete regional failure and failure of the shared DynamoDB service remain outside the recovery boundary.

Keywords: cloud architecture, assessment continuity, failover, writer fencing, DynamoDB transactions, idempotency, durable receipts, AWS Step Functions.

Contents

Acknowledgement i

Abstract ii

Abbreviations and Notation vii

1 Introduction 1
- 1.1 Project Context . . . 1
- 1.2 Problem Statement . . . 1
- 1.3 Objectives . . . 2
- 1.4 Scope and Recovery Boundary . . . 2
- 1.5 Principal Contributions . . . 2
- 1.6 Motivating Failure Sequence . . . 2

2 Background and System Analysis 3
- 2.1 Availability and Correctness . . . 3
- 2.2 Relevant Cloud Mechanisms . . . 3
- 2.3 Comparison of Recovery Approaches . . . 4
- 2.4 Stakeholders and Functional Requirements . . . 4
- 2.5 Quality Requirements . . . 5
- 2.6 Design Decisions and Their Consequences . . . 6

3 Cloud Architecture and Recovery Design 7
- 3.1 Architectural Overview . . . 7
- 3.2 Application and Routing Components . . . 8
- 3.3 Network Design . . . 8
- 3.4 Liveness and Application Readiness . . . 8
- 3.5 Confirmation Policy . . . 9
- 3.6 Recovery Workflow . . . 10

4 Data Design and Correctness Mechanisms 11
- 4.1 Logical Data Model . . . 11
- 4.2 Writer Epoch and Atomic Fencing . . . 11
- 4.3 Request Identity and Payload Binding . . . 13
- 4.4 Atomic Submission Transaction . . . 13
- 4.5 Attempt Timing and Receipt Replay . . . 14
- 4.6 Transactional Outbox and Audit Reconciliation . . . 15

5 Implementation and Application Workflows 16
- 5.1 From Architecture to a Working Assessment . . . 16
- 5.2 Teacher Preparation and Publication . . . 17
- 5.3 Learner Entry and Timed Answering . . . 19
- 5.4 Final Submission and Receipt Retrieval . . . 21
- 5.5 Code Structure and Deployment . . . 21

6 Experimental Evaluation and Discussion 23
- 6.1 Questions and Evidence . . . 23
- 6.2 Backend Verification . . . 23

6.3 Local Fault Decisions . . . 24
6.4 AWS Experiment Design . . . 26
6.5 Recovery Timeline and Writer Authority . . . 27
6.6 Resolving the Lost Response . . . 29
6.7 Reconciliation: Correctness and Work Performed . . . 29
6.8 Stale-Writer Validation . . . 30
6.9 Interpretation and Validity . . . 30

7 Security, Operations, and Cost **31**
7.1 Security Design . . . 31
7.2 Separation of Service Permissions . . . 31
7.3 Threat and Failure Considerations . . . 32
7.4 Operational Lifecycle . . . 32
7.5 Monitoring Delay and Activity Models . . . 33
7.6 Cost Model . . . 34
7.7 Alignment with Cloud Architecture Principles . . . 35

8 Limitations, Future Work, and Conclusion **36**
8.1 Current Limitations . . . 36
8.2 Future Work . . . 37
8.3 Conclusion . . . 37

References **38**

A Reproducibility Guide **39**
A.1 Local Execution . . . 39
A.2 Backend Verification Commands . . . 39
A.3 Controlled Recovery Demonstration . . . 39
A.4 Evidence and Operating Boundaries . . . 40
A.5 Rebuilding the Report and Graphs . . . 40

List of Figures

3.1 Logical cloud architecture and regional recovery services 7
3.2 Verified recovery sequence and safety branches 10

4.1 Submission and exact receipt replay contract 14

5.1 Teacher home and classroom overview 16
5.2 Assessment policy settings 17
5.3 Question authoring and grading configuration 18
5.4 Published assessment management 18
5.5 Learner assessment catalogue 19
5.6 Assessment lobby and start policy 19
5.7 Timed answering and review controls 20
5.8 Continuing a saved attempt 21
5.9 Retrieved local submission receipt and feedback 22

6.1 Local suppression when both backends are unready 25
6.2 Local recovery to backend B at epoch 2 25
6.3 Local recovery event history 26
6.4 Recorded AWS event timeline and timing windows 27
6.5 Authoritative writer epoch transition 28
6.6 Recorded AWS recovery cockpit 28
6.7 Observed reconciliation scans and new audit work 30

7.1 Analytical monitoring and confirmation delay 33
7.2 Analytical scheduled monitoring activity 34

List of Tables

2.1 Architectural approaches and their relationship to the assessment problem . . . 4
2.2 Functional requirements and implemented mechanisms 5
2.3 Decisions that determine the recovery contract 6

3.1 Component placement and responsibility 9
3.2 Readiness decision policy for the active and standby backends 9

4.1 Principal logical records in the assessment recovery path 11
4.2 Ordering of an in-flight write and a conditional promotion 12

5.1 Implementation boundaries and the contracts they protect 22

6.1 Evidence layers and their limits 23
6.2 Current-source backend verification 24
6.3 Recorded submission and replay comparison 29

7.1 Selected risks and implemented controls 32
7.2 Relationship to the Well-Architected pillars 35

Abbreviations and Notation

Term	Meaning
ACID	Atomicity, consistency, isolation, and durability
API	Application programming interface
ASL	Amazon States Language
AWS	Amazon Web Services
AZ	Availability Zone
CDN	Content delivery network
EC2	Amazon Elastic Compute Cloud
HTTP	Hypertext Transfer Protocol
IAM	AWS Identity and Access Management
JWT	JSON Web Token
NAT	Network address translation
OAC	Origin access control
RPO	Recovery point objective: acceptable loss of committed data
RTO	Recovery time objective: target time to restore an operation
S3	Amazon Simple Storage Service
SSM	AWS Systems Manager
TTL	Time to live
VPC	Virtual private cloud
(b, e)	Active backend identifier and monotonically increasing writer epoch
r	Persistent application submission request identifier
$H(a)$	Deterministic hash binding a request to its answer payload and scope

Chapter 1

Introduction

1.1 Project Context

Online assessments require more than the availability of a webpage. During a timed examination, the important operation is the student's ability to preserve answers and obtain confirmation of a final submission. An application may appear alive while database access is unavailable. A response may disappear after a valid submission has already committed. A recovering instance may continue processing an old request after another instance has become the active writer.

These situations connect infrastructure recovery with application correctness. Simply redirecting traffic does not establish which instance is allowed to write, whether an answer has been stored, or whether a retry will create a duplicate. Continuum addresses this connection through an assessment application and an observable recovery controller.

The platform provides student and teacher workflows, timed attempts, server autosave, final submission receipts, classroom assignment, and a recovery cockpit. Its architectural focus is a bounded HTTP assessment path that can recover from an application-instance failure while preserving the original attempt and submission identity.

1.2 Problem Statement

The project investigates the following problem:

> How can a timed assessment platform recover from failure of its active application instance while preserving a single authorised writer, continuity of the learner's attempt under unchanged assessment policy, and a reliable outcome for a submission whose response was lost?

Four conditions make this problem significant. First, process liveness can disagree with the ability to serve assessment operations. Second, changing a route does not automatically stop requests already executing on the former backend. Third, a network error does not reveal whether the server committed a request. Fourth, downstream audit processing may be interrupted even after the assessment write succeeds.

An effective design therefore needs a recovery decision, a write-authorisation mechanism, a retry contract, and a way to complete pending secondary work. These mechanisms must agree on the meaning of success.

1.3 Objectives

The principal objectives are to:

1. Implement teacher authoring, timed learner attempts, progress saving, and final receipts in a usable application.
2. Recover from sustained application failure through verified promotion, while rejecting obsolete writers at the database transaction.
3. Resolve lost responses through exact receipt replay, complete audit work without duplication, and evaluate these contracts locally and in AWS.

1.4 Scope and Recovery Boundary

The supported AWS design uses two application instances in separate Availability Zones of Sydney. Both instances share one regional DynamoDB table. Managed routing, monitoring, and orchestration components also remain dependencies of the assessment path. The demonstrated fault is loss or application-specific unavailability of one backend while these shared services remain accessible.

The project does not claim recovery from a complete AWS Region outage, a regional DynamoDB outage, or every possible Availability Zone failure. Two-AZ placement provides infrastructure separation, but stopping one instance is a narrower experiment than disabling an entire zone. Local mode is narrower still: it runs separate A/B HTTP servers inside a single Go process and shares an in-memory store.

Collaborative boards and WebSocket features exist in the broader application. The bounded cloud recovery deployment serves ordinary HTTP assessment and control routes; it does not expose the board WebSocket path or demonstrate board failover.

1.5 Principal Contributions

Continuum connects verified routing, transactional writer authority, exact receipt replay, and recoverable audit work. Health checks establish service capability; receipts and canonical records establish the submission outcome.

1.6 Motivating Failure Sequence

Near an attempt's end, the learner submits answers, the database commits, and the response is lost. The active application then fails. The learner is uncertain about both service availability and the submission, although the database already holds the result.

Recovery selects a capable backend; receipt lookup resolves the particular request. This distinction prevents a transport error from being interpreted as a failed submission and motivates independent record inspection in the experiment.

Chapter 2

Background and System Analysis

2.1 Availability and Correctness

Availability concerns whether a requested operation can be performed. Durability concerns whether an acknowledged result remains stored. Idempotency concerns whether repeating a logical request changes its outcome. A recovery architecture must consider all three when a client cannot distinguish a failed request from a lost response.

For example, a student sends a final submission and receives a gateway error. The backend may have failed before committing, or the transaction may have succeeded before the response was dropped. Automatically generating a new submission identifier would discard the evidence connecting the retry to the original operation. Continuum retains that identifier and the submitted answer snapshot.

Writer fencing solves a different ambiguity: which instance may commit after promotion? A backend can remain reachable long enough to finish an earlier request. A database condition tied to the current epoch allows the storage system to reject that stale mutation even if the backend itself has not stopped.

2.2 Relevant Cloud Mechanisms

Amazon DynamoDB supports all-or-nothing transactional operations with conditional checks on distinct items. These operations provide the storage primitive for applying a writer condition alongside the assessment records [1]. Continuum's application receipt is a persistent data model; it does not depend only on the short-lived idempotency token associated with the database API.

AWS Step Functions Standard workflows provide durable execution history and explicit retry and error paths [2]. In Continuum, those capabilities organise the recovery sequence. Correctness still depends on conditional storage updates and idempotent application tasks, because the workflow deliberately includes retries.

Gateway VPC endpoints allow the private router and verifier to reach S3 and DynamoDB without a NAT device [3]. Public endpoint verification and AWS control API calls are handled by functions outside the application VPC. This placement follows the connectivity requirements of the existing implementation.

2.3 Comparison of Recovery Approaches

Table 2.1: Architectural approaches and their relationship to the assessment problem

Approach	Useful capability	Assessment concern remaining
Single backend	Simple deployment and data path	Instance failure interrupts the application
Process restart	Restores a failed process	Does not resolve a response lost after commit
Health-based load balancing	Distributes or redirects traffic using health checks	Request identity and stale writes still require application controls
Active–active application tier	Multiple serving instances	Concurrent mutations require a clear data consistency policy
Continuum active–standby design	Conditional promotion, writer fencing, receipt replay	Shared regional services remain dependencies; capacity is bounded

The custom router is appropriate for studying explicit writer ownership in a small prototype. It also introduces an extra Lambda invocation, an authoritative-record read, proxy timeouts, and additional implementation responsibility. The project therefore treats it as a bounded architectural choice rather than a general substitute for managed load balancing.

2.4 Stakeholders and Functional Requirements

The learner needs to start an assessment, answer questions, resume an existing attempt, understand save status, and obtain a receipt. The teacher needs to author and publish assessments, assign them to classrooms, configure availability and attempt limits, and review results. The operator needs to inspect active ownership, observe health, inject controlled faults, and examine recovery events.

Table 2.2: Functional requirements and implemented mechanisms

ID	Requirement	Mechanism
FR1	Enforce timed student attempts	Persisted start time and server deadline validation
FR2	Resume saved answers	Attempt records, browser state, and server autosave
FR3	Confirm final submission	Durable request-bound receipt and result view
FR4	Recover active instance failure	Readiness confirmation and Step Functions promotion
FR5	Reject obsolete writers	Backend/epoch condition in write transactions
FR6	Prevent conflicting request replay	Answer payload hash and attempt-scoped receipt lookup
FR7	Complete audit work safely	Atomic outbox creation and idempotent reconciliation
FR8	Restrict operational controls	Separate administrator token and private backend ingress

2.5 Quality Requirements

2.5.1 Safety

Only a writer matching the authoritative backend and epoch may commit protected application changes. A changed answer payload must not reuse a committed request identifier. A failed public smoke test or unresolved reconciliation conflict must not be labelled a successful recovery.

2.5.2 Continuity

An application switch must not restart the attempt clock. Under unchanged assessment metadata, both backends calculate the same deadline from the persisted start time. A browser buffer preserves unsynchronised work but does not constitute a durable acknowledgement. The precise effect of an administrator changing duration during an attempt is discussed in Chapter 4.

2.5.3 Observability and Reproducibility

The operator should be able to examine the active backend, epoch, liveness, readiness, suppression reason, verification result, and reconciliation result. Local tests and deployment artifacts should reproduce the contracts without requiring a continuously running cloud stack.

2.5.4 Deployment Constraints

The deployment keeps all regional resources in Sydney (`ap-southeast-2`). It excludes cross-Region replicas and Lambda@Edge, and avoids NAT gateways, always-on load balancers, and purchased domains.

2.6 Design Decisions and Their Consequences

Table 2.3: Decisions that determine the recovery contract

Decision	Reason	Consequence
Regional shared data	Survives an application-instance replacement without copying student data	DynamoDB remains a shared dependency
One authoritative writer	Makes ownership explicit during an uncertain failover	Standby capacity does not increase normal write throughput
Fence inside each transaction	Closes the gap between route lookup and commit	Every protected mutation path must use the condition
Persisted request receipt	Resolves commits whose responses are lost	Client identity, payload binding, and record retention are required
Three readiness observations	Avoids reacting to an isolated transient error	Confirmation deliberately adds delay
Public smoke after promotion	Checks the route used by the browser	A deep-health success must be complemented by submission-specific validation

These choices are evaluated against the assessment operation, rather than by the number of AWS services used. Managed services provide the storage and execution primitives; the application defines how their outcomes combine into an accepted submission.

Chapter 3

Cloud Architecture and Recovery Design

3.1 Architectural Overview

Continuum separates the student-facing request path from the recovery decision path. The request path serves the application using one authorised backend. The decision path observes both backends, confirms an incident, and changes authority through a conditional update. DynamoDB connects these paths through a shared control record and the assessment data.

The static frontend is stored in a private S3 bucket and distributed through CloudFront using origin access control. The browser calls a separate HTTPS API Gateway endpoint for application data. The default API integration invokes the router Lambda; protected status and action routes invoke the control Lambda. Figure 3.1 shows the logical dependencies of the supported deployment.

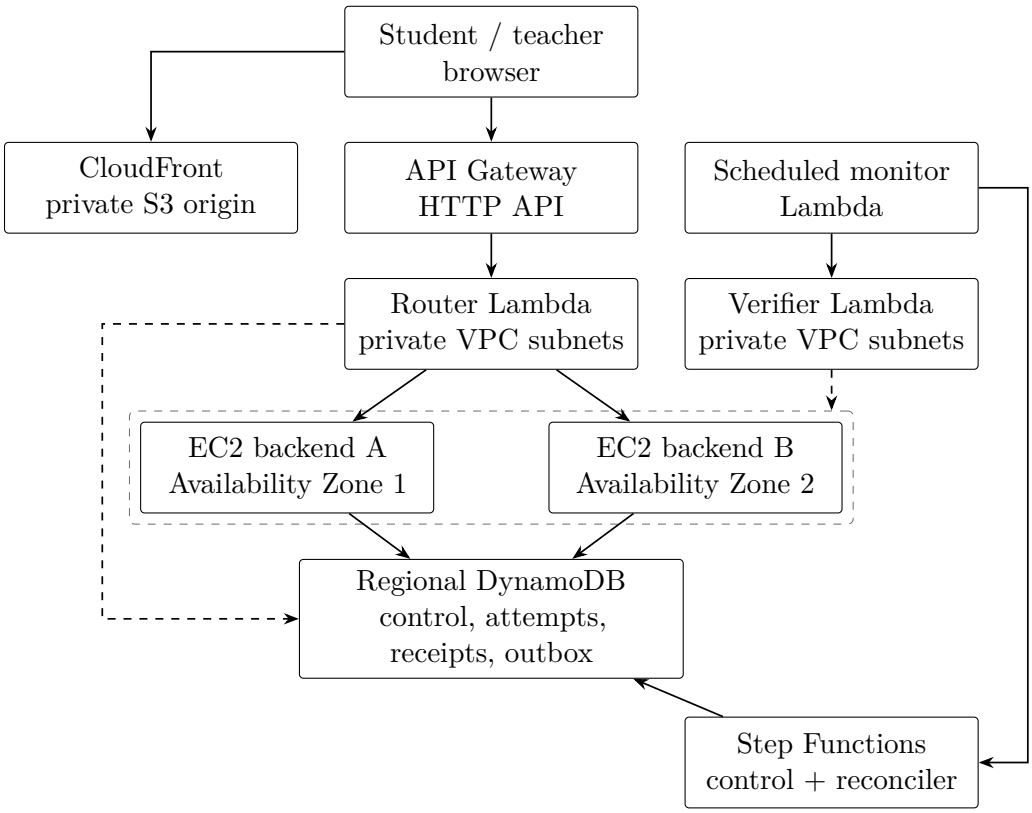


Figure 3.1: Logical cloud architecture. Regional services run in Sydney; CloudFront provides global content delivery. Dashed paths show the router's control-record read and the verifier's probes of the grouped A/B tier.

3.2 Application and Routing Components

The two EC2 instances run the Go application with a shared regional store. They are assigned identities A and B. A strongly consistent read of the active control record provides the router with the current backend and epoch. The router supplies those values as internal request metadata when it forwards a request over the private application address.

The record is stored at partition key `CONTROL#ACTIVE` and sort key `STATE`. Its important fields are `active_backend`, `epoch`, and `paused`. An epoch identifies a particular grant of writer authority, rather than simply identifying a machine. Returning authority to A after a recovery must therefore use a newer epoch than A held previously.

There are two layers of protection. Request middleware propagates writer context into application operations, and storage transactions validate that context against the current control record. The second layer is essential: a route can change between the router's read and the backend's database write.

3.3 Network Design

The VPC contains application and Lambda subnets across two Availability Zones. Application instances use public subnets and automatic public IPv4 addresses for bootstrap and Systems Manager connectivity. Their application port is nevertheless private in use: inbound TCP port 8000 is allowed only from the router/verifier security group, and no inbound SSH rule is created.

The router and verifier use private Lambda subnets. S3 and DynamoDB gateway endpoints are associated with the relevant route tables. No NAT gateway is provisioned. Functions that call public AWS APIs or perform the public smoke test remain outside the application VPC. Connecting a Lambda function to a public subnet would not itself provide internet access [4]; the architecture instead assigns functions according to their actual network dependencies.

3.4 Liveness and Application Readiness

Liveness answers whether the process responds. Deep readiness answers whether it can currently serve the assessment path under the configured checks. The probe calls `/health/live` and `/health/deep` separately with bounded timeouts. Deep health includes database capability and configured host/application checks.

The partial-failure scenario injects a database-access stall into one application. Its process remains alive, but readiness becomes false. This tests a useful distinction without claiming that the DynamoDB service itself failed. When both backends fail readiness, the monitor suppresses promotion because switching to the standby would not establish a usable target.

Table 3.1: Component placement and responsibility

Component	Placement	Responsibility
Frontend	S3 and CloudFront	Static application delivery
Router	Private VPC Lambda subnets	Authoritative route read and HTTP forwarding
Verifier	Private VPC Lambda subnets	Probe A/B and apply protected fault controls
Application A/B	Separate EC2 Availability Zones	Assessment and classroom HTTP operations
Monitor	Lambda outside the VPC	Repeated incident confirmation and workflow start
Control	Lambda outside the VPC	Lease, promotion, public smoke, instance controls
Reconciler	Lambda outside the VPC	Bounded outbox processing
Data store	Regional DynamoDB	Shared durable application and control state

Table 3.2: Readiness decision policy for the active and standby backends

Active ready	Standby ready	Recovery decision
Yes	Yes	Keep the current writer; no failure is established
Yes	No	Keep the current writer and restore standby capability
No	Yes	Confirm sustained failure, acquire the lease, and recheck before promotion
No	No	Suppress promotion; a usable target has not been established

Readiness observations are evidence for a decision, not permanent guarantees. The candidate is checked again while the incident lease is held because its condition may change after the monitor's last observation. Liveness remains useful for distinguishing a stopped HTTP server from a responding process with a failed dependency.

3.5 Confirmation Policy

The cloud monitor is triggered by an EventBridge scheduled rule once per minute. Within a monitor invocation, the policy requires three observations separated by five seconds. These observations must refer to the same active backend and epoch. A route change, a paused control plane, recovered active readiness, or an unhealthy standby prevents automatic promotion.

Three samples contain two configured waiting intervals, giving ten seconds of delib-

erate spacing before probe and invocation overhead. The schedule can also introduce delay before the first sample. Consequently, the policy does not imply a ten-second fault-to-recovery guarantee.

3.6 Recovery Workflow

After confirmation, the Standard workflow acquires an incident lease and performs a fresh verification under that lease. The conditional promotion checks the expected backend, expected epoch, live lease ownership, and unpaused state. The target backend receives the next epoch.

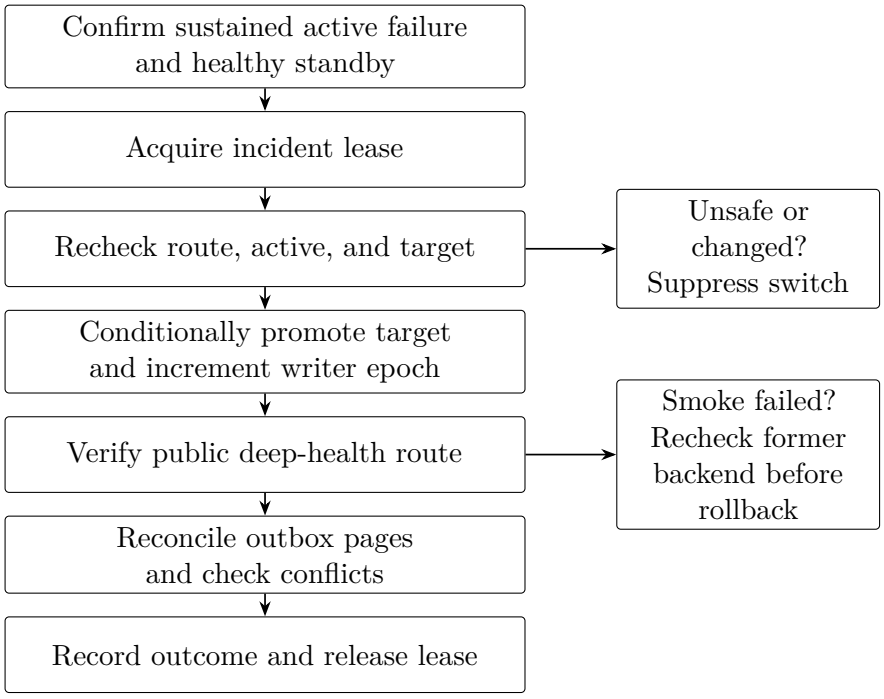


Figure 3.2: Recovery sequence and the principal safety branches. The workflow records a successful outcome only after the required verification and reconciliation checks.

The public smoke test traverses the externally reachable API and checks deep health through the selected route. A failed smoke result cannot be converted into a successful recovery. Rollback requires fresh readiness of the former backend and advances the epoch again; the controller cannot safely restore an old authority value.

Reconciliation is a separate phase. It scans pending journal work in bounded pages, verifies canonical submission records, and records completion only after the required checks succeed. Manual reconciliation also runs asynchronously through a workflow, so the public action request does not wait for the full backlog.

Lease ownership uses explicit expiry timestamps in conditional operations, independent of asynchronous DynamoDB TTL cleanup. Error paths release the lease; expiry bounds ownership if execution stops.

Chapter 4

Data Design and Correctness Mechanisms

4.1 Logical Data Model

Continuum uses a single DynamoDB table with partition and sort keys to represent distinct entity types. The application store manages users, classroom membership, assessment metadata, attempts, submissions, receipts, and outbox entries. Control and audit namespaces support the recovery machinery.

Table 4.1: Principal logical records in the assessment recovery path

Record	Important information	Purpose
Active control	Backend, epoch, paused state	Authoritative route and writer identity
Incident lease	Owner and explicit expiry	Serialise recovery ownership
Assessment	Questions, duration, windows, policy	Define delivery and grading rules
Attempt	Learner, answers, start, version, attempt number	Resume work and enforce lifecycle
Submission	Immutable attempt snapshot and latest reference	Retain the accepted answer outcome
Request receipt	Request ID, payload hash, scope, commit time	Resolve exact retries
Outbox journal	Target submission, request ID, processed flag	Recover interrupted audit processing
Audit summary	Request ID and canonical target reference	Record one validated downstream outcome

The current implementation supports attempt-scoped submission history and a latest-submission reference. The recorded October cloud rehearsal predates some broader product additions; its counts refer to the canonical submission and recovery records inspected for that particular request. Logical submission counts should not be confused with the number of physical records used by a newer schema.

4.2 Writer Epoch and Atomic Fencing

Let the authoritative control state be

$$C = (b, e, p), \tag{4.1}$$

where b is the active backend, e is its epoch, and p is the paused flag. A request carries the writer identity $W = (b_w, e_w)$. A protected mutation is permitted only if

$$b_w = b \quad \wedge \quad e_w = e \quad \wedge \quad p = \text{false}. \tag{4.2}$$

The database evaluates this condition in the same `TransactWriteItems` operation as the application changes. A preliminary read alone would be insufficient, because a promotion could occur between that read and the commit.

Consider a request routed to A at epoch 1. If promotion commits B at epoch 2 before the request's write transaction, condition (4.2) fails for A. If A's transaction commits first, its data belongs to the preceding valid writer interval. This establishes an ordering at the storage boundary. It does not require the failed process to receive a notification or stop immediately.

Failback and rollback obey the same rule. Returning to A after B does not reuse epoch 1; it advances to a newer epoch. An old request therefore cannot become valid again merely because its original backend name is active once more.

4.2.1 The Race Between a Submission and Promotion

The decisive event is the database commit order. Table 4.2 considers a request already executing on A at epoch 1 while the controller attempts to promote B. A routing header identifies the proposed writer; it is not by itself proof of continuing authority.

Table 4.2: Ordering of an in-flight write and a conditional promotion

Ordering	Storage outcome	Client implication
A/1 transaction commits first	A's records belong to a valid epoch-1 writer interval	A lost response can be resolved from the durable receipt
B/2 promotion commits first	A/1 fails the writer condition; the protected transaction does not commit	Retry through the current route using the retained request identity
Promotion response is lost	A retry is checked against the same lease and expected next epoch	Repeated control work must not advance authority twice for one operation
A later becomes active again	A receives a newer epoch	An old A/1 request remains invalid

The guarantee concerns protected writes using the shared control condition. It does not turn every application read into a global serialisable snapshot, and it does not recover a regionally unavailable data store.

4.3 Request Identity and Payload Binding

Each final submission has a persistent request identifier r. The browser retains the identifier together with the immutable answer snapshot being submitted. The server binds that request to its assessment, learner, attempt, and payload through a deterministic hash. Conceptually,

$$K_r = (r, \text{assessment}, \text{learner}, \text{attempt}, H(a)). \tag{4.3}$$

A retry of the same logical request retrieves the original receipt. Reusing r with changed answers or a mismatched scope is a conflict. This prevents a response-lost retry from silently becoming a different answer submission.

The application receipt is stored durably and contains the committed timestamp, request identifier, version, score, total, and attempt number. Its lifetime and lookup semantics belong to the application data model. They are distinct from a temporary database-client request token.

4.4 Atomic Submission Transaction

The current store builds the final submission using one transaction containing the following writes and condition:

1. Create the immutable submission snapshot for the attempt.
2. Update the latest-submission reference.
3. Create the request receipt if that request has not already committed.
4. Create the pending outbox journal entry.
5. Mark the active attempt submitted using version and attempt-number conditions.
6. Check the authoritative backend, epoch, and unpaused state.

The attempt condition prevents an outdated version from completing a conflicting lifecycle transition. The uniqueness conditions prevent duplicate receipt and snapshot creation. Atomicity ensures that the application does not acknowledge a final submission without its corresponding receipt and recovery journal.

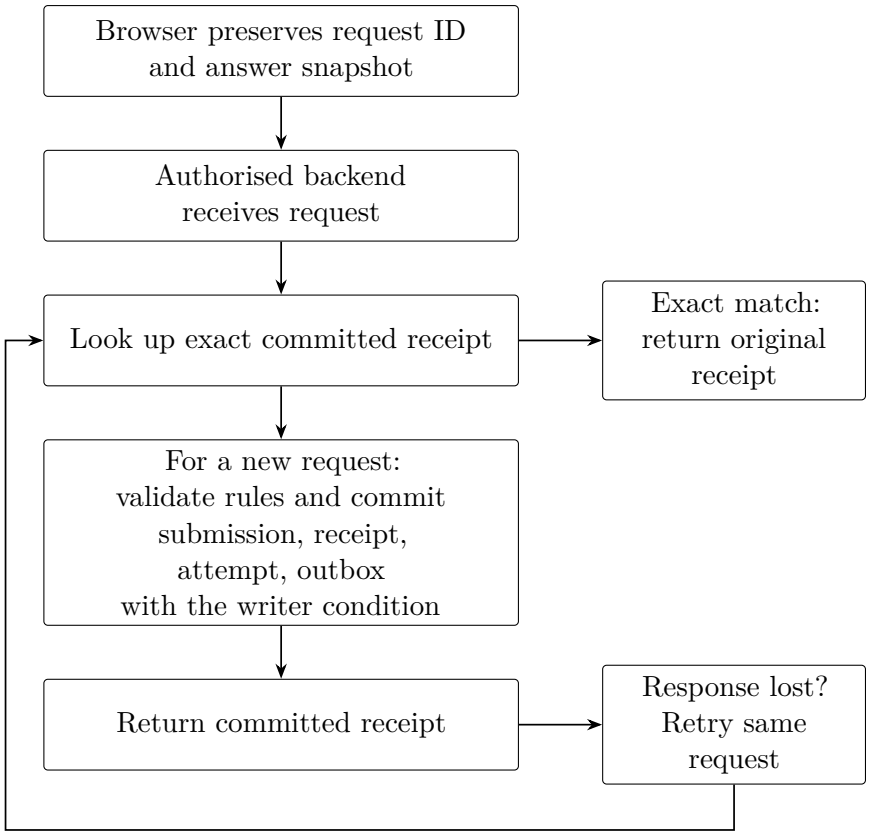


Figure 4.1: Submission and receipt replay contract. An exact retry follows the lookup path; a changed payload under the original identifier is rejected.

4.5 Attempt Timing and Receipt Replay

The server persists the attempt start time. In the current store, the effective attempt deadline is recalculated from that start and the current assessment duration:

$$D_{\text{attempt}} = t_{\text{start}} + 60\,000\, d_{\text{current}}, \tag{4.4}$$

where timestamps are in milliseconds and duration d is in minutes. A backend change does not recreate the attempt, so the deadline remains the same when the assessment metadata is unchanged. The fresh demonstration assessment has a 15-minute duration and one attempt.

This is a narrower property than freezing assessment policy at attempt creation. Changing the duration while an attempt is running changes the calculated deadline. Availability-window checks and late-work settings are also evaluated separately. The service checks the attempt deadline before calling the store, with a default five-second grace period; the write transaction checks the attempt version and writer authority, but does not independently compare commit time with a frozen deadline. These details define the current timing semantics and motivate policy snapshots and commit-boundary tests for institutional deployment.

For a new submission, the service enforces the configured rules and its bounded

server-side grace policy. An exact committed receipt is looked up before rejecting the request because the exam has since closed. Retrieving proof of a previous commit does not constitute accepting new answers after the deadline. If no matching receipt exists, ordinary deadline checks still apply.

The browser presents local buffering separately from confirmed storage. An answer retained on the student's device is useful for retry, but it cannot establish that the server accepted the final submission. A prolonged outage may outlast the student's remaining permitted time; the architecture does not promise recovery of every uncommitted offline answer.

4.6 Transactional Outbox and Audit Reconciliation

An outbox entry is created atomically with the submission. It describes pending downstream work and points to the canonical submission. The reconciler reads pending entries, checks that the target and request identity agree, and creates an answer-free audit summary while marking the journal processed.

These operations are conditional and atomic. Repeated reconciliation therefore does not add another logical audit outcome for the same request. A matching pre-existing audit can repair a pending journal only after the canonical target and complete key are checked. Conflicts are reported rather than overwritten.

The journal may carry answer information needed by the implementation; the resulting audit summary omits the answers. This distinction matters for access control and retention. The reconciler is not a general-purpose message broker or a proof of exactly-once processing for arbitrary external systems.

4.6.1 Interruption Points and Recovery

If execution stops before the submission transaction commits, there is no accepted final outcome from that transaction. If it stops after commit but before returning a response, the receipt is already present. If it stops before audit reconciliation, the pending journal remains discoverable. If reconciliation is retried after completion, its conditions identify the existing logical result.

This design removes the unsafe gap in which a submission could commit without the journal describing its required audit work. It also prevents a secondary audit step from becoming part of the learner's synchronous request latency. The trade-off is that reconciliation must scan or otherwise locate pending work, monitor conflicts, and retain records long enough to repair interruptions.

These properties depend on every protected mutation using the writer condition, exact replay retaining its payload-bound receipt, and the regional store remaining available. They establish a bounded assessment contract rather than exactly-once behaviour for arbitrary external systems.

Chapter 5

Implementation and Application Workflows

5.1 From Architecture to a Working Assessment

The recovery mechanism is integrated with an application that teachers and learners can use. A teacher prepares and publishes a classroom assessment; a learner enters it, answers against a server-managed attempt, and receives a submission reference. These operations exercise authentication, authorisation, persistence, grading, and recovery through the same public API.

The screens in this chapter were captured from the running application on 23 October 2026 using a synthetic BCSE355L classroom and a five-question, fifteen-minute assessment [14]. They show local mode: two backend HTTP servers share an in-memory store within one process. They establish the implemented user workflow; the separate AWS experiment in Chapter 6 supplies evidence for cloud recovery.

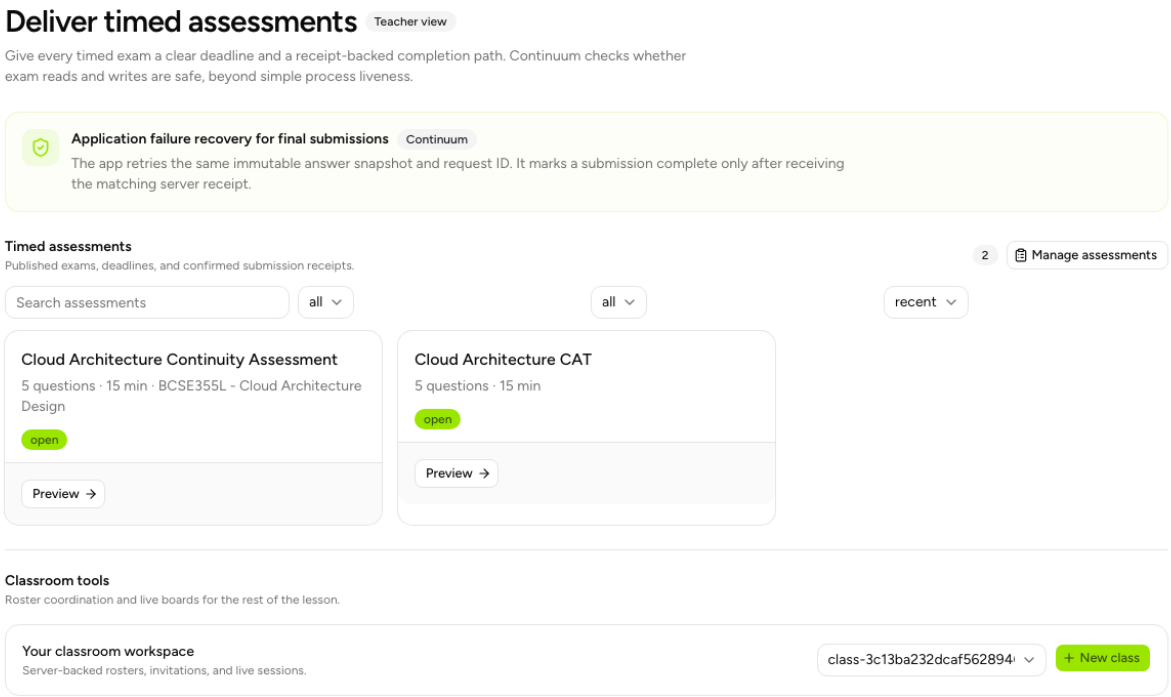


Figure 5.1: Teacher home with classroom context and assessment actions, using synthetic course data.

5.2 Teacher Preparation and Publication

The editor captures the title, classroom, instructions, duration, and attempt limit. Each question has options, a correct choice, points, and feedback. The example uses five two-point questions, making the maximum score ten.

A draft permits preparation before learners see the assessment. Publishing exposes it through the catalogue. Published questions are protected against changes to an existing grading basis; availability and instructions remain separately managed [9].

Classroom membership and API role checks protect assessment access and management. Retakes create a new attempt context while preserving earlier receipts.

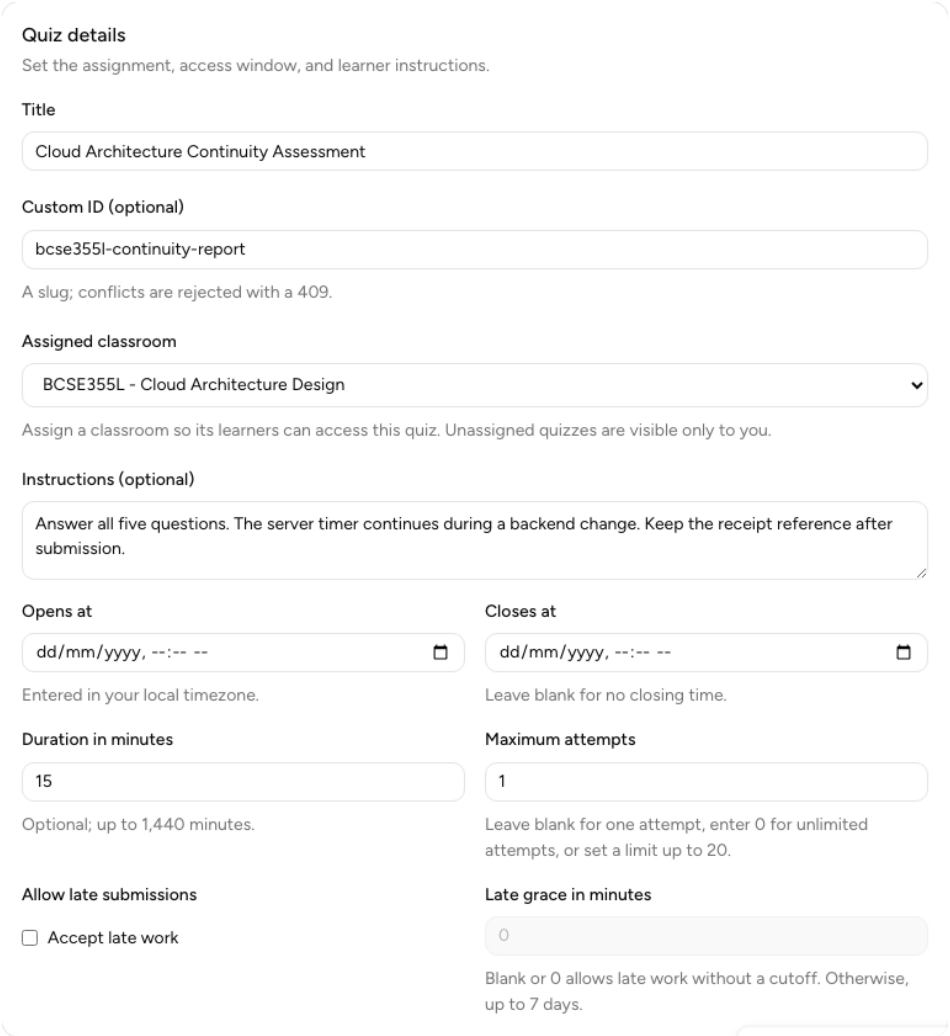


Figure 5.2: Assessment policy: classroom assignment, instructions, a fifteen-minute duration, and one allowed attempt.

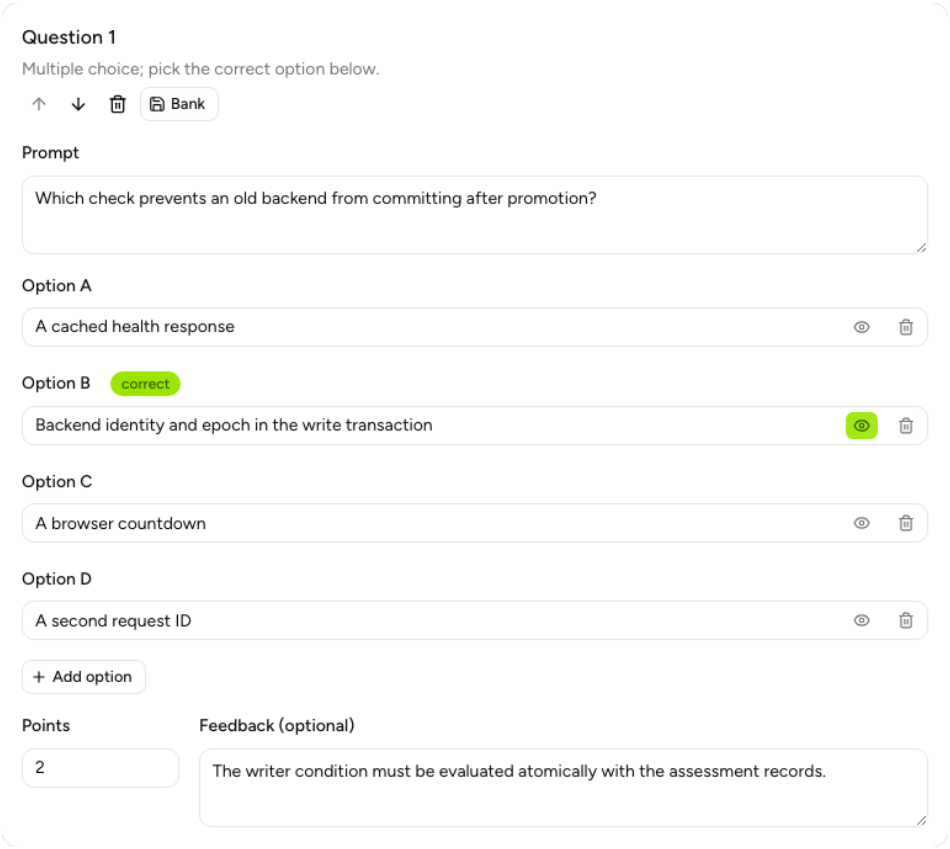


Figure 5.3: Question authoring with a selected correct option, explicit points, and explanatory feedback.

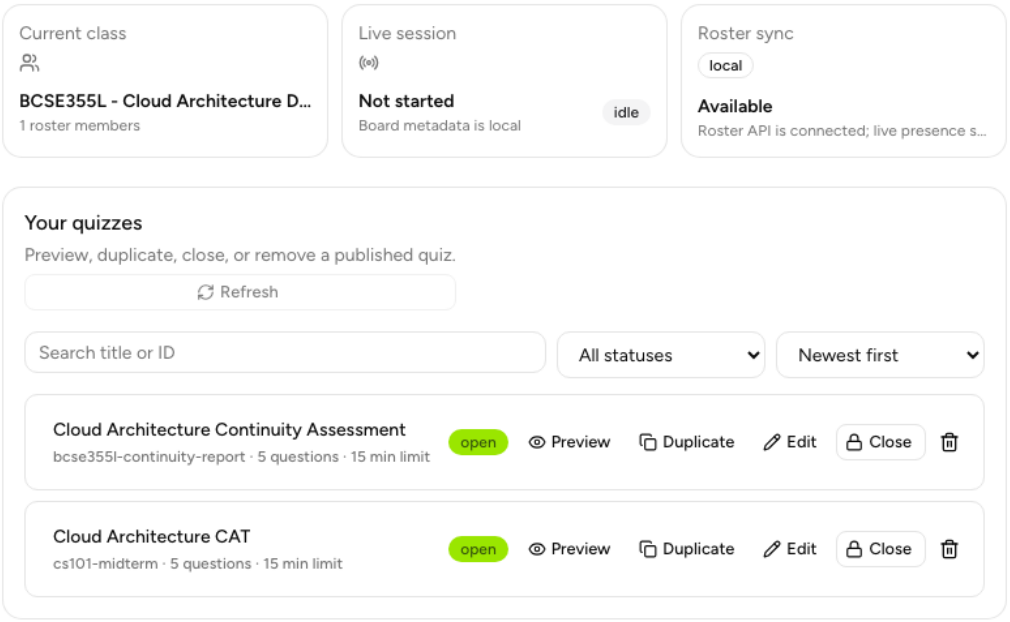


Figure 5.4: Teacher management view after publication, with availability and lifecycle controls.

5.3 Learner Entry and Timed Answering

After authentication, the learner sees assessments permitted by classroom membership. The catalogue supports discovery; the lobby explains the selected assessment before the learner starts. Starting obtains the server-managed attempt and its persisted start time. Changing backend does not create another start event.

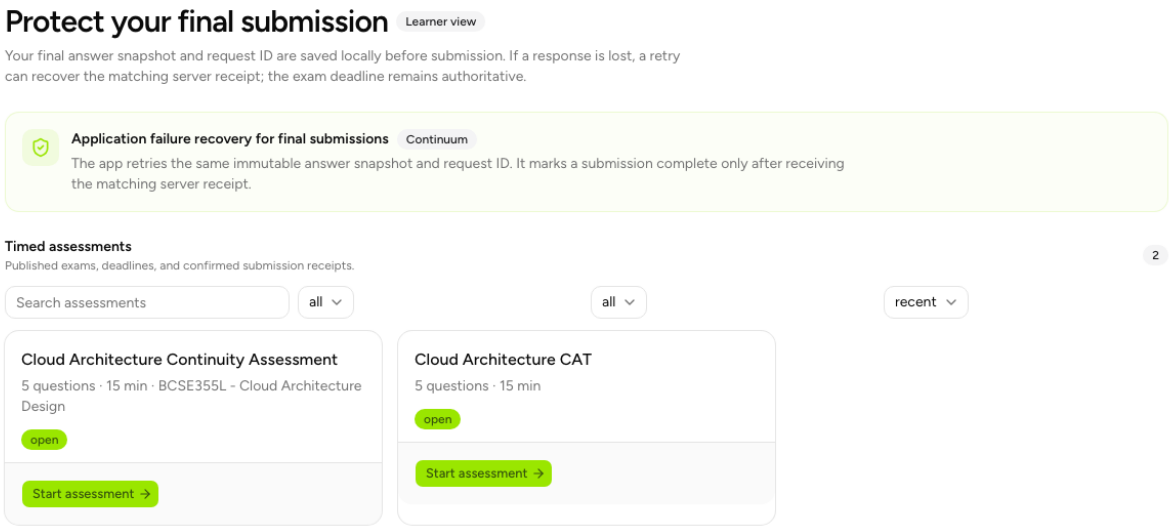


Figure 5.5: Learner catalogue containing the published Cloud Architecture Continuity Assessment.

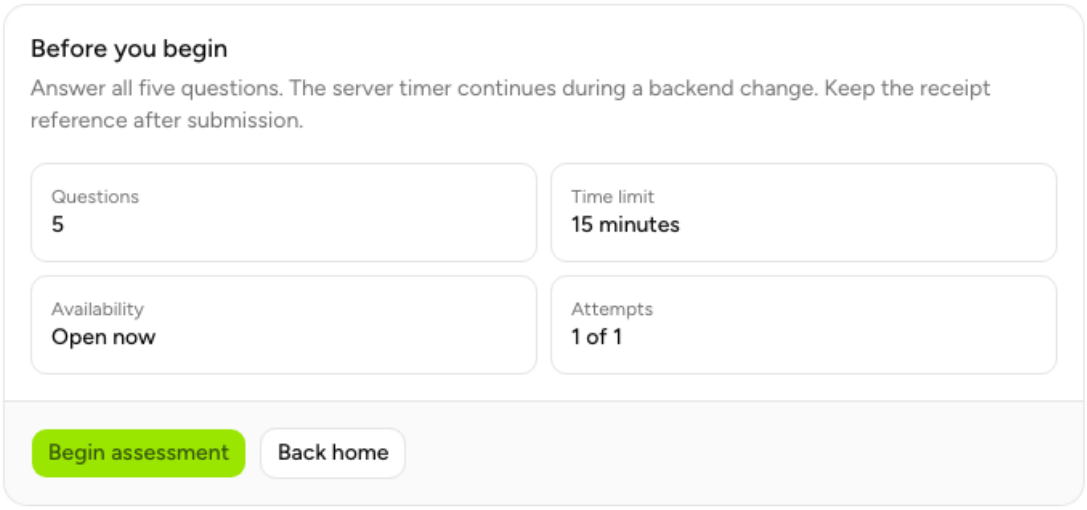


Figure 5.6: Assessment lobby showing instructions and the policy before the timed attempt begins.

Within the assessment, navigation separates answering from review. A learner can select an answer, flag a question, revisit unanswered questions, and inspect the remaining time before submission. These controls help the learner identify what remains to be done without creating a second attempt during an interruption.

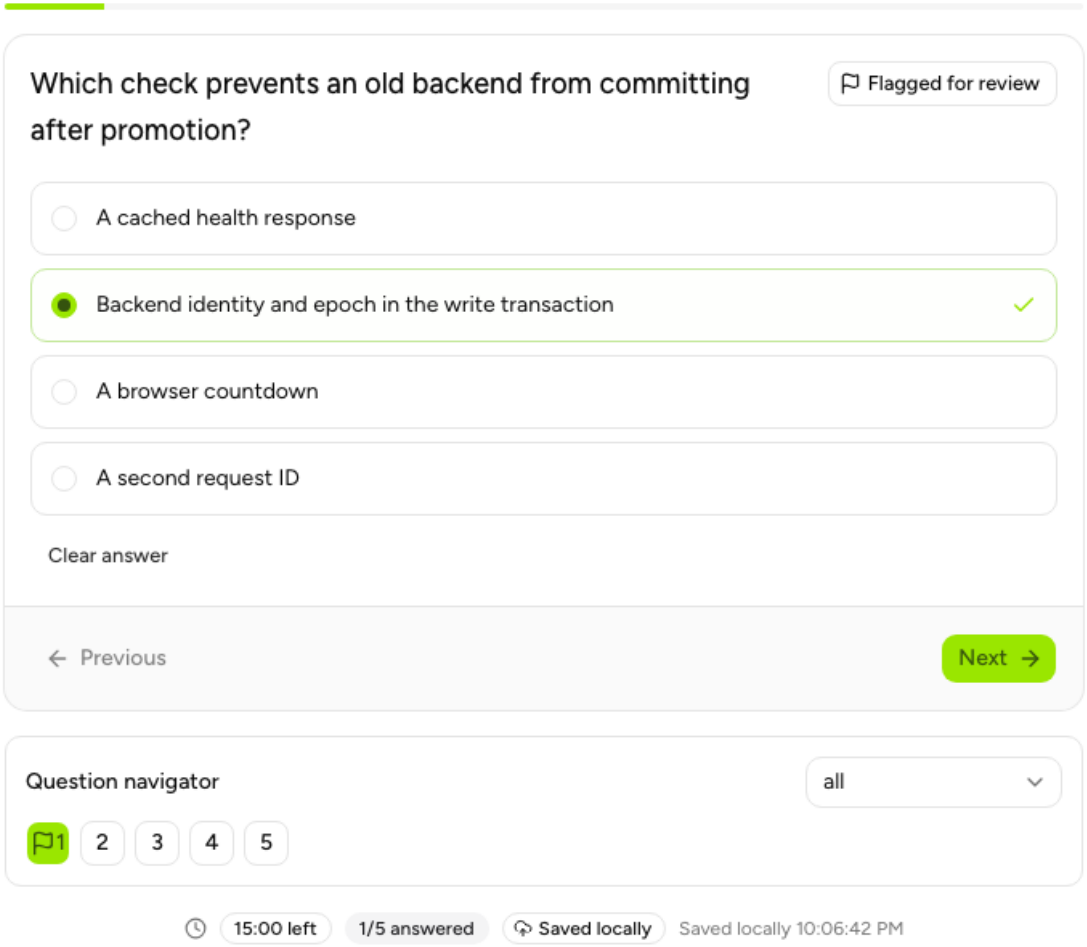


Figure 5.7: Timed answering with a selected choice, review flag, question navigation, remaining time, and local-save status.

5.3.1 Three Different Meanings of Saved

Browser storage, acknowledged progress, and a committed final result are separate states. "Saved locally" means this browser retained progress; a successful save response acknowledges backend attempt state. Only a final receipt identifies the immutable submitted snapshot and committed outcome.

Figure 5.8 demonstrates continuation from device state, rather than durability after browser loss. Backend changes preserve the attempt start time. Its deadline also depends on the current duration setting; Section 4.5 explains that policy boundary.

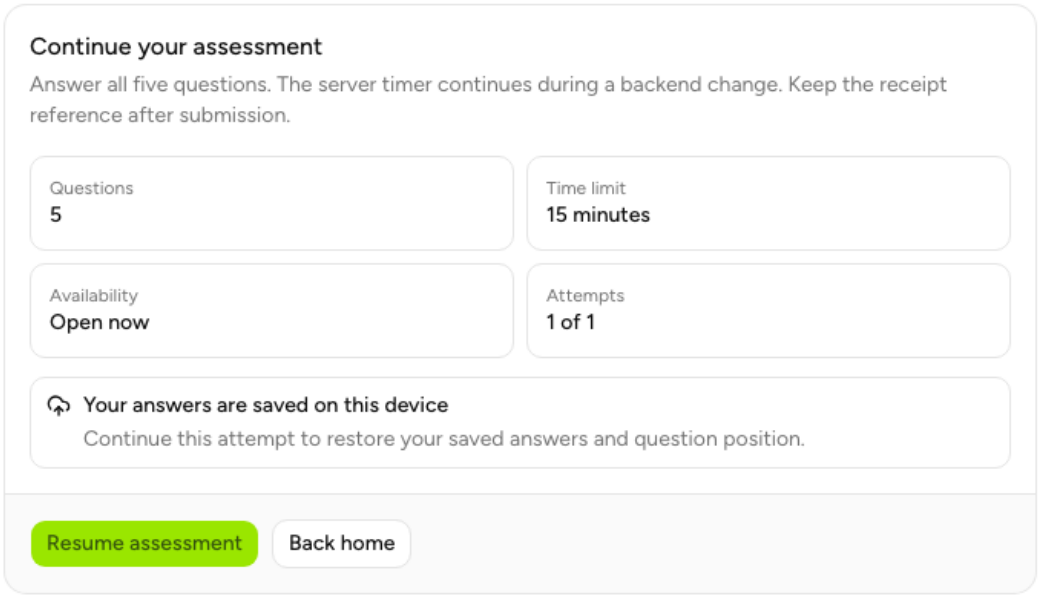


Figure 5.8: Continuation prompt after revisiting the assessment. The interface identifies answers retained on this device.

5.4 Final Submission and Receipt Retrieval

Before final submission, the browser preserves a request identifier with a fixed answer snapshot. Bounded retries use this identity rather than generating a different logical submission for every network attempt. A transport error may leave the outcome unknown, so the interface resolves the receipt before asking the learner to create a new submission.

Figure 5.9 shows the completed synthetic attempt: score, submission time, receipt reference, copy/download actions, and question feedback. This local capture illustrates receipt retrieval through the application. It is not the response-loss experiment; equality of the original and replayed AWS commit is established independently in Chapter 6.

5.5 Code Structure and Deployment

Go implements the backend; React and TypeScript implement the interface. TanStack Router and Query coordinate navigation and server state, while Vite and Tailwind CSS support the frontend build. The AWS implementation uses the Go SDK v2 and Terraform. Table 5.1 identifies the boundaries that make the recovery contract reviewable.

Build scripts package application and Lambda artifacts deterministically. Bootstrap retrieves a private bundle and encrypted runtime configuration. A full deployment can replace instances and therefore requires a maintenance window. Pause/resume preserves the desired instance and monitoring state; private deployment files remain outside the report package.

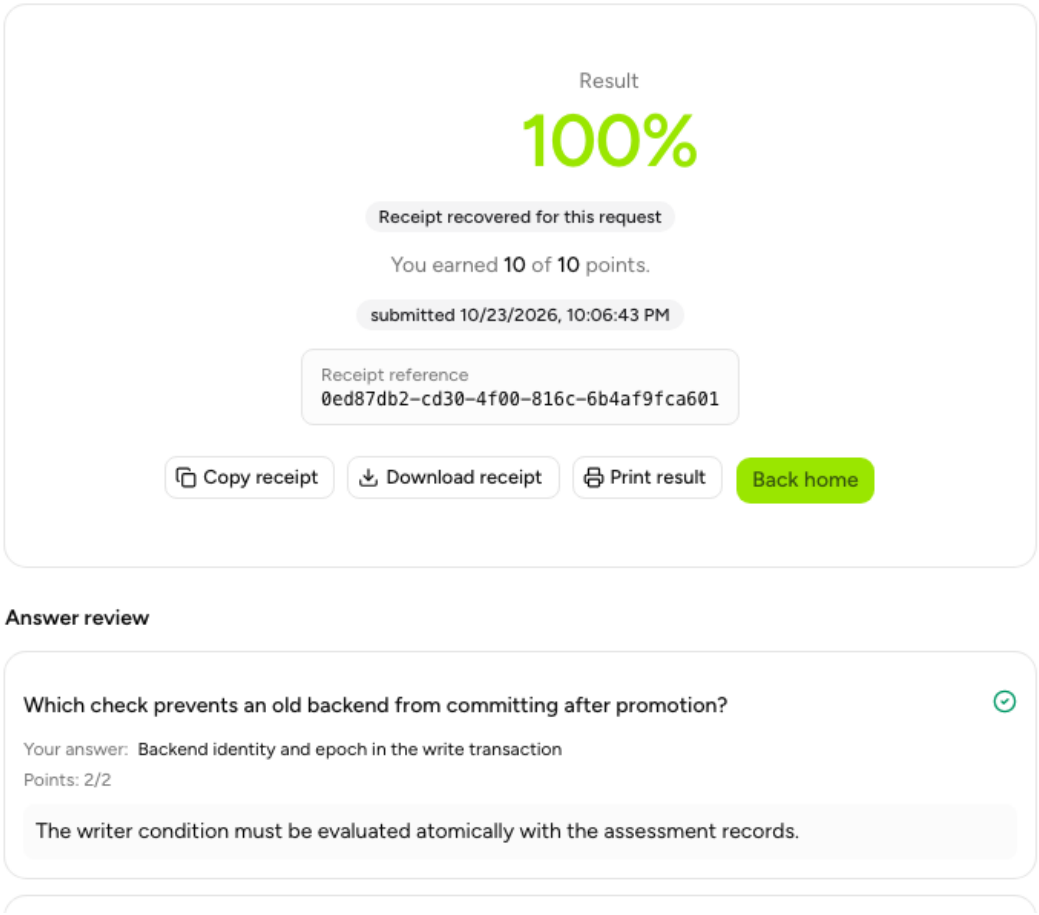


Figure 5.9: Retrieved local result with a receipt reference, submission time, and answer review. The synthetic learner earned ten of ten points.

Table 5.1: Implementation boundaries and the contracts they protect

Source boundary	Responsibility
`internal/httpapi`	Authentication, classroom access, validation, and response mapping
`internal/exams`	Availability, attempt timing, saving, retakes, and submission rules
`internal/store`	Memory/DynamoDB persistence; receipts, attempts, transactional writes
`internal/clusterstate`	Authoritative backend/epoch contract for protected writes
`internal/continuum`	Router, capability probes, monitoring, control state, and events
`cmd/demo-reconciler`	Conditional outbox processing and audit completion
`statemachine/demo-failover.asl.json`	Recovery sequence, verification, and error paths
`infra/demo`	Supported single-Region AWS deployment
`web/src`	Role workflows, pending requests, and receipt presentation

Chapter 6

Experimental Evaluation and Discussion

6.1 Questions and Evidence

The evaluation asks four questions: does recovery restore a usable public route; can the learner resolve a submission whose response was lost; does an obsolete backend lose write authority; and does repeated reconciliation avoid another logical audit result? Each requires different evidence. A green health indicator answers only the first; request comparison and database inspection are needed for the second and fourth.

The evidence comprises current-source backend checks, fresh local interface and fault captures on 23 October 2026, and the preserved AWS rehearsal of 10 October 2026. Local mode verifies behaviour through real HTTP servers, but its shared process and memory store do not reproduce EC2 isolation or DynamoDB durability. AWS observations apply to the recorded deployment and synthetic workload [9, 14, 11, 12].

Table 6.1: Evidence layers and their limits

Layer	Useful evidence	Important limit
Source and tests	Conditions, timing rules, race cases, replay and conflicts	Cannot establish a deployed cloud outcome alone
Local application	Role workflows, probe decisions, HTTP recovery, receipt display	Shared process and non-durable memory
AWS rehearsal	Actual instance stop, promoted route, replay, record inspection	One bounded run, not a latency distribution

6.2 Backend Verification

On 23 October 2026, six existing Go package suites passed with the race detector enabled. This package-level result does not imply that every production workload or fault ordering has been tested. Appendix A retains the commands.

Table 6.2: Current-source backend verification

Package	Coverage focus	Result
`internal/continuum`	Routing, promotion, leases, readiness, events	Passed
`internal/store`	Writer conditions, receipts, attempts, durable entities	Passed
`cmd/demo-reconciler`	Reconciliation and conflict handling	Passed
`integration`	Real HTTP response loss, recovery, exact replay	Passed
`internal/httpapi`	Validation, access, lifecycle, receipts	Passed
`internal/exams`	Assessment rules, timing, attempts	Passed

The preserved implementation review separately records frontend checks, production builds, Terraform validation, and state-machine validation [10]. Those historical results complement the backend suites; they were not additional checks rerun during screenshot capture.

6.3 Local Fault Decisions

The first local sequence disabled application readiness on both backends while their liveness probes still passed. The screen retained A at epoch 1 and recorded suppression (Figure 6.1). This is the intended response: routing to another unready process cannot restore the operation the learner needs.

After clearing the faults, the active A HTTP server was stopped. Three consecutive failed probes led to B becoming active at epoch 2. The local screen reports 10,010 ms from the first failed monitor probe to route verification. This excludes the earlier wait for that first probe and is separate from the AWS recovery measurement.

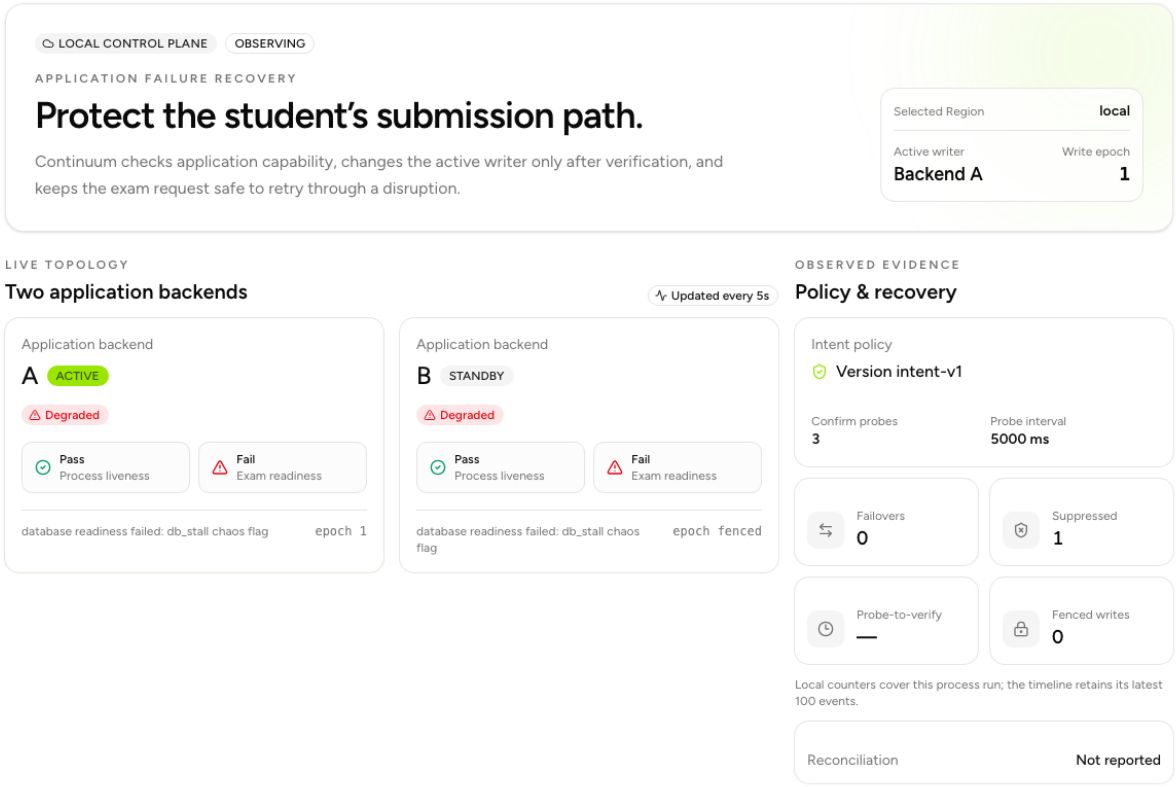


Figure 6.1: Local unsafe-standby case: both processes are alive but fail assessment readiness. Promotion is suppressed and A retains epoch 1.

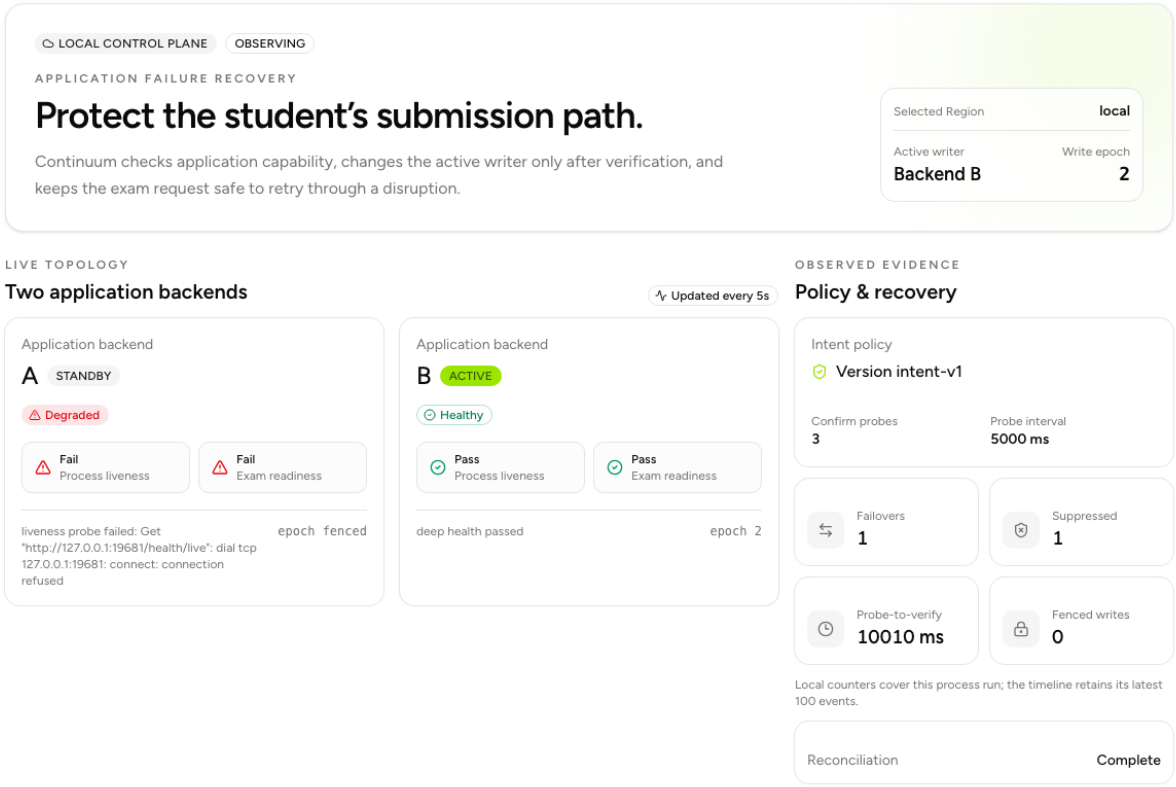


Figure 6.2: Local recovery after stopping A's HTTP server: B is healthy and active at epoch 2; reconciliation is complete.

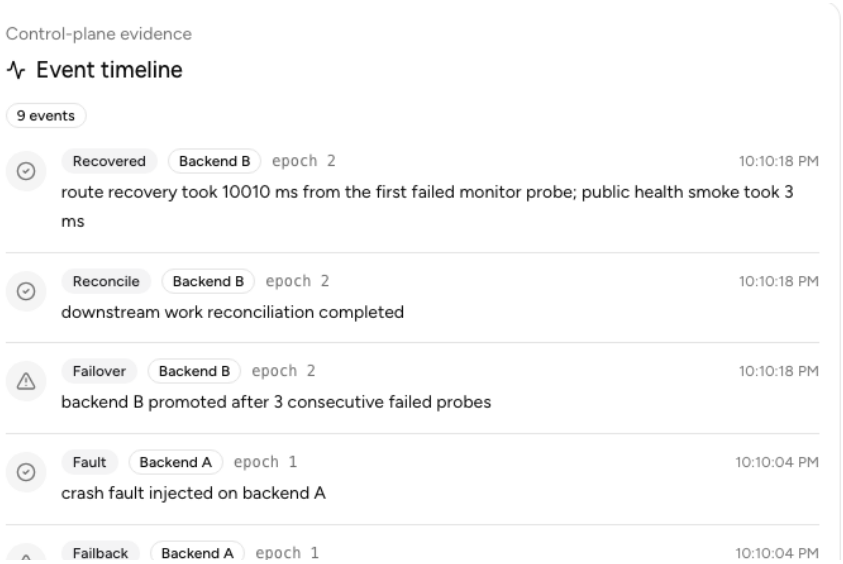


Figure 6.3: Event history records the injected fault, promotion, public verification, and reconciliation. The list is ordered for operational viewing.

6.4 AWS Experiment Design

The cloud rehearsal used two EC2 application instances in separate Sydney Availability Zones, a shared regional DynamoDB table, and the deployed routing and recovery services. A was initially active at epoch 1. Before the principal run, a 7.5-second transient fault recovered during confirmation, and an unsafe-standby case made both backends unready. Both suppressed promotion and retained epoch 1 [11].

The principal sequence combined a committed submission with an unavailable response:

1. Submit a fixed synthetic answer snapshot with a persistent request identifier. Commit succeeds, but injected response loss returns HTTP 503.
2. Request an actual stop of the active EC2 instance. Allow the scheduled monitor and workflow to determine the recovery action.
3. Observe B at epoch 2, verify public deep health, and inspect reconciliation.
4. Replay the unchanged request, then alter its payload under the same identifier and inspect the resulting conflict.
5. Independently count submission-related records and repeat reconciliation.

This resolves one ambiguous committed outcome through a backend failure. It does not measure cohort throughput or continued writes during loss of DynamoDB.

6.5 Recovery Timeline and Writer Authority

Figure 6.4 aligns events to the start of the instance-stop action. The workflow-start event appears at 41.940 seconds; promotion at 45.287 seconds, public verification at 45.470 seconds, and reconciliation at 47.970 seconds. External polling observed recovery at 50.472 seconds. These timestamps establish the sequence without identifying every internal scheduling or network delay.

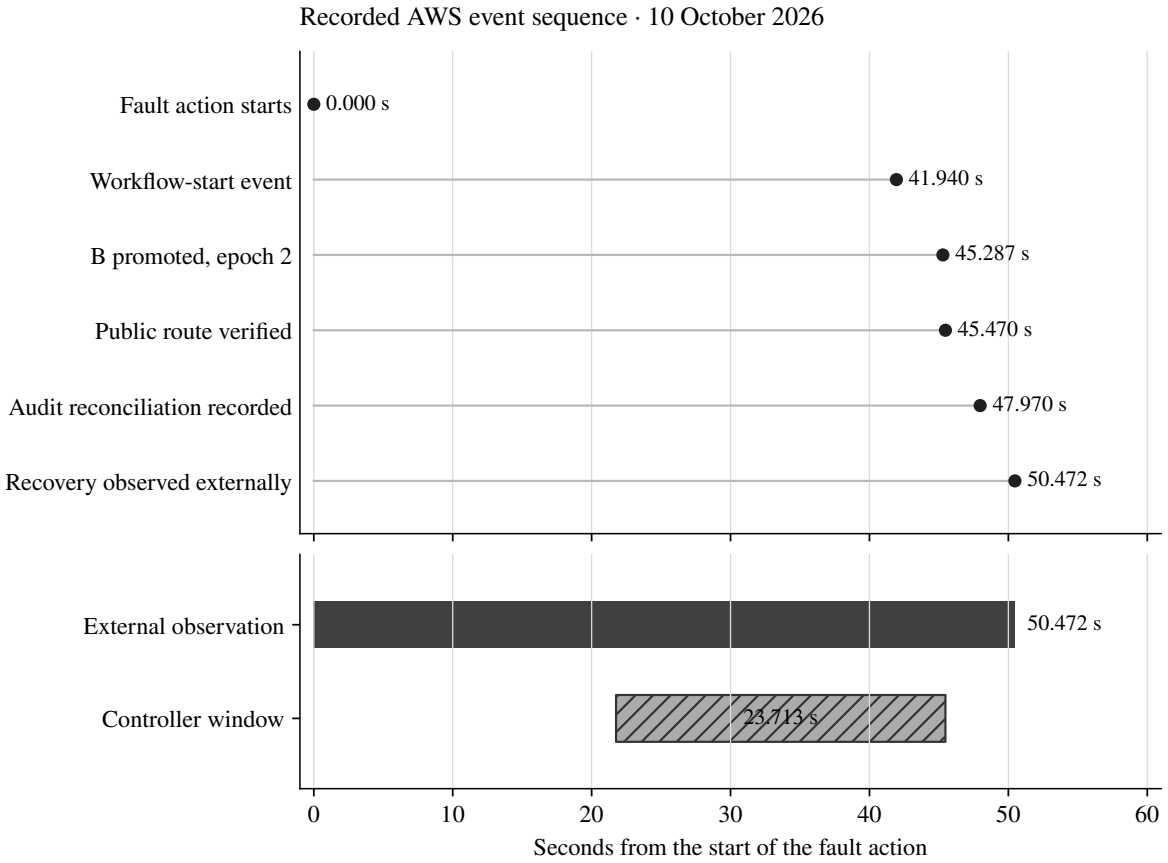


Figure 6.4: Recorded AWS event sequence and timing windows. The controller window's left endpoint is derived from the verification timestamp minus its recorded 23.713-second duration.

The external and controller intervals use different boundaries:

$$T_{\text{external}} = t_{\text{external observation}} - t_{\text{fault action}} = 50.472\,\text{s}, \tag{6.1}$$

$$T_{\text{controller}} = t_{\text{public smoke}} - t_{\text{check cycle start}} = 23.713\,\text{s}. \tag{6.2}$$

The controller interval begins before the workflow-start event because it includes monitoring and confirmation. Subtracting the durations does not isolate a single scheduling overhead: the endpoints differ, and external observation also includes polling and reconciliation. The preserved workflow result supplies the controller duration [12].

Figure 6.5 represents authority, not merely process availability. An old in-flight transaction either commits before the promotion condition wins or fails the writer condition

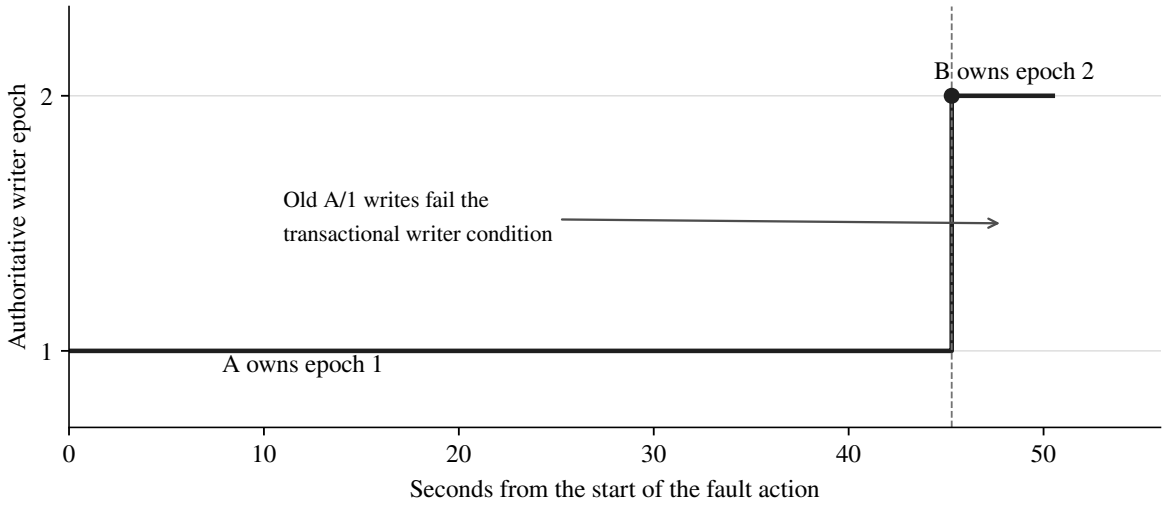


Figure 6.5: Writer state reconstructed from the initial record and promotion event. Authority changes once from A/1 to B/2; restarting A does not grant it a newer epoch.

after it. Routing convergence alone would leave a period in which old A could still mutate shared data. The transactional fence closes that gap for protected writes.

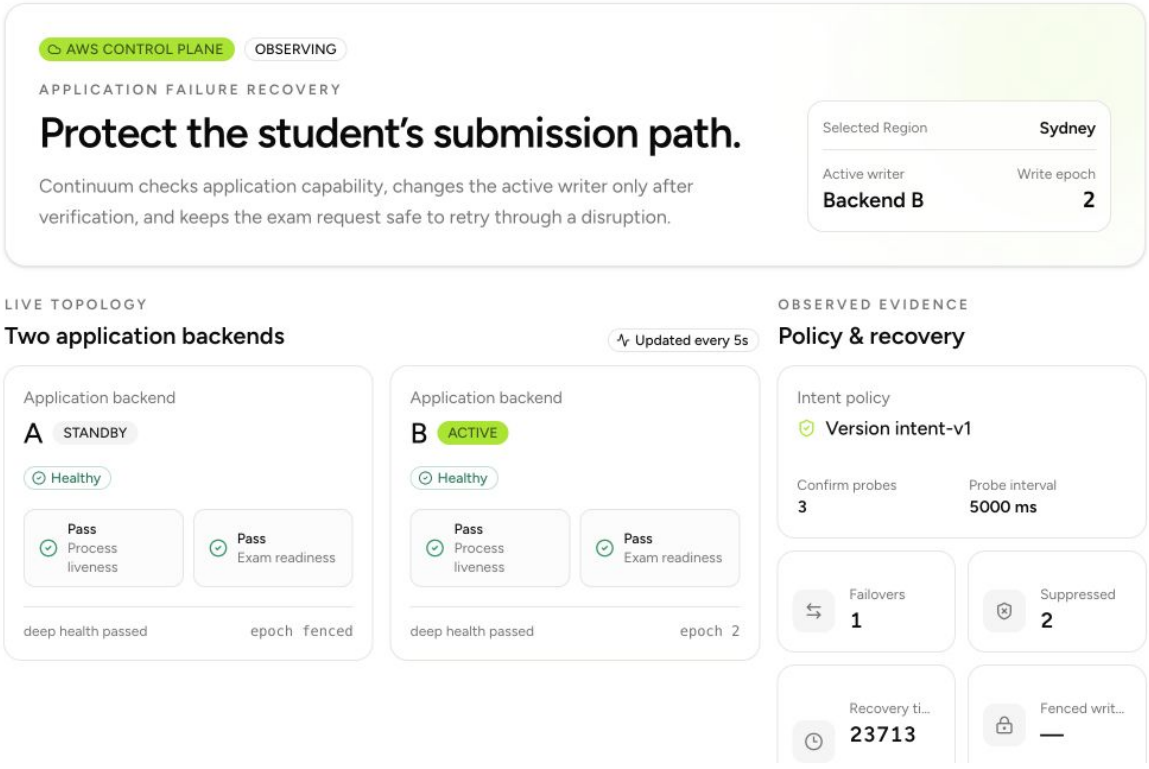


Figure 6.6: Preserved AWS cockpit from the cloud rehearsal, showing B active at epoch 2. This capture is separate from the new local demonstrations.

6.6 Resolving the Lost Response

The commit timestamp precedes the fault action by 1.609 seconds. Although the original request returned HTTP 503 after response loss, the exact retry returned a duplicate receipt with the original commit time. Here, duplicate means that the existing result was recovered, not that another submission was accepted.

Table 6.3: Recorded submission and replay comparison

Field or operation	Recorded outcome
Persistent request identifier	`cloud-6fbb0f1bc6fd4a86bdab2e9c17f33d23`
Original response after commit	HTTP 503 (deliberate response-loss injection)
Original commit timestamp	`1791652428408` milliseconds since the Unix epoch
Exact replay	`duplicate`; identical identifier and commit timestamp
Receipt scope and result	Attempt 1, version 1, score 2 of 5
Same identifier, changed answers	HTTP 409 conflict
Independent record inspection	One canonical submission, one receipt, one processed journal entry, one audit summary

The unchanged identifier and commit time answer the learner's question: the original submission exists despite the failed response. The changed-payload conflict establishes that the identifier cannot silently acquire another answer snapshot. Neither result follows merely from promoting a healthy standby.

6.7 Reconciliation: Correctness and Work Performed

The automatic recovery execution succeeded with one processed entry and zero conflicts. A repeated execution succeeded with zero new processed entries and zero conflicts. Independent inspection found one audit summary for the request [12]. Together, these support one logical downstream outcome for this inspected submission.

The repeat still scanned 32 items. Idempotency prevents repeated business effects but does not make discovery free. The reconciler scans a table using an outbox filter, so `ScannedCount` can grow with unrelated data. A pending-work index or dedicated stream is a scaling improvement; this experiment does not establish that repeated scans remain efficient at institutional scale.

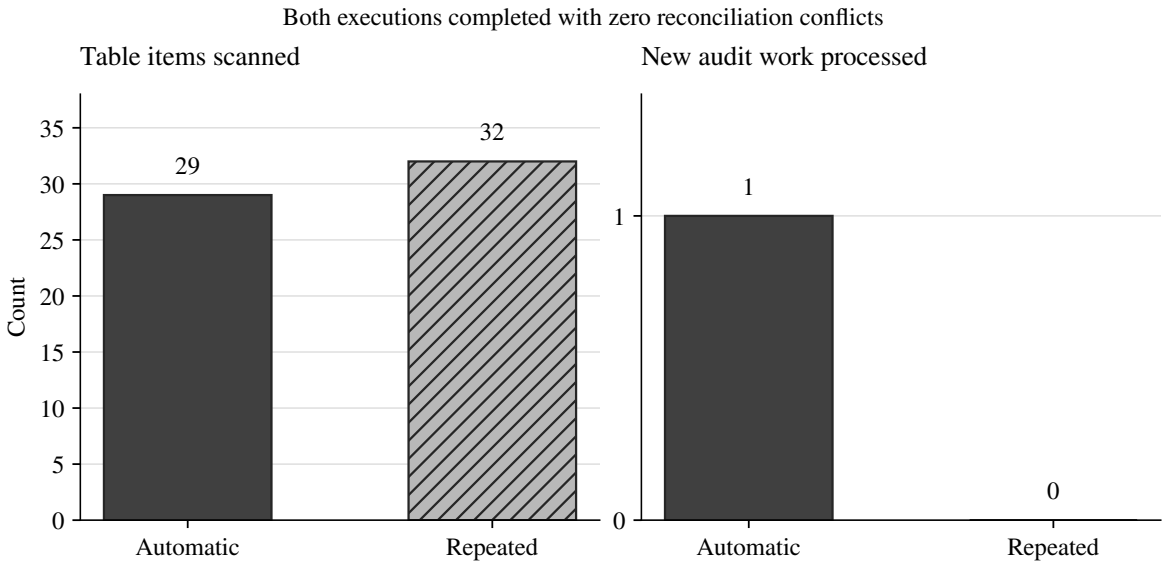


Figure 6.7: Observed reconciliation: 29 and 32 table items scanned, with one and zero new audit outcomes. Scan count does not mean pending-submission count.

6.8 Stale-Writer Validation

After A restarted, a synthetic registration sent directly to A with epoch 1 returned HTTP 503 and `WRITER_FENCED`. A strongly consistent email-index lookup found no created record [13]. This checks stale registration specifically. Exam-submission fencing is covered by transactional storage and API tests; a stale exam submission was not sent directly to AWS in this rehearsal.

6.9 Interpretation and Validity

The strongest result combines route recovery, unchanged submission identity, and one inspected logical audit result. Suppression cases matter equally: a useful controller sometimes preserves the route rather than promotes. The local screens make decisions visible, while the AWS records establish the bounded cloud outcome.

Four limits constrain generalisation. One cloud run cannot establish mean or percentile recovery time, a production RTO, or an SLA. The workload is synthetic rather than a concurrent examination cohort. Shared DynamoDB, control services, and the selected Region remain dependencies. Finally, current-source checks and later local captures are dated separately from the preserved deployment. No data loss was observed for the inspected committed request, but this is not an unconditional zero-RPO guarantee.

Chapter 7

Security, Operations, and Cost

7.1 Security Design

The public browser interfaces use HTTPS through CloudFront and API Gateway. The API authenticates application users and applies role and classroom access checks. Passwords are hashed, and student sessions do not grant access to fault injection. The recovery cockpit requires a separate administrator control token.

Network policy restricts application ingress to the router/verifier security group on port 8000. The deployment does not open inbound SSH; Systems Manager supports instance management. Backend traffic within the VPC uses HTTP in the prototype, which makes the private network boundary significant. Additional service-to-service transport protection is a production hardening requirement.

Runtime service roles provide AWS credentials to EC2 and Lambda. Bootstrap configuration is stored in an encrypted Systems Manager parameter, and the application bundle is retrieved from a private S3 bucket. Private runtime files, deployment outputs, and Terraform state must be protected because they can contain operational secrets.

7.2 Separation of Service Permissions

The application role can access the table for permitted application operations, including conditional checks on the active record. Explicit policy restrictions prevent it from writing the control and audit namespaces. The control function manages the active writer and instance operations, while the reconciler owns its bounded audit-processing responsibility.

The monitor is configured without the fault-control token. The workflow role invokes the functions needed for the recovery sequence. This distribution limits the authority of a component that only needs to observe or forward requests. The model concerns service permissions; human team-member access remains managed through the AWS project experience.

7.3 Threat and Failure Considerations

Table 7.1: Selected risks and implemented controls

Risk or failure	Control and remaining boundary
Unauthorised fault action	Separate administrator token; credential protection remains essential
Forged or obsolete writer context	Private ingress plus database-enforced backend/epoch condition
Concurrent recovery actions	Incident lease, expected-state conditions, retry-safe promotion
Changed answers under one request ID	Scope and payload binding with conflict rejection
Misleading healthy process	Deep readiness and healthy-standby confirmation
Interrupted audit processing	Atomic outbox and conditional reconciliation
Private data in operational artifacts	Private state/output files; synthetic demonstration data
Shared regional dependency failure	Suppress unsafe promotion; outside the demonstrated recovery scope

The prototype is not presented as an institutional examination service ready for unrestricted use. Formal privacy review, retention policy, broader abuse controls, external security testing, backup/restore exercises, and operational ownership would be needed before using real examination data at scale.

7.4 Operational Lifecycle

Development and most rehearsals can use local mode. Cloud resources are enabled for bounded demonstration windows. The supported lifecycle is deploy, resume, verify both backends, perform the controlled scenario, capture the outcome, and pause the stack.

Before a cloud deployment, the selected Region is confirmed through AWS Settings, under View all projects, Overview, Additional Info, Region. The supported Terraform configuration validates Sydney and rejects another regional value. The project plan and current spend status are checked through AWS Settings and the plan API before resources are started. The recorded rehearsal used the Free plan; this historical fact does not establish a later credit balance.

Pause stops the EC2 instances and disables the monitoring schedule. Existing executions can finish, and retained disks, data, logs, or objects can continue to consume

credits. Destruction is a separate action that removes persistent data as well as compute, so required synthetic results must first be retained deliberately.

7.5 Monitoring Delay and Activity Models

Monitoring frequency creates a direct trade-off between nominal detection delay and recurring work. The implemented scheduled monitor runs once per minute. Three confirmation probes have two five-second gaps, so even a fault discovered immediately requires a nominal ten seconds of confirmation spacing. Figure 7.1 explores one-, two-, and five-minute schedules; only the one-minute rate is the current deployment.

Let S be the schedule period in seconds, $\phi \in [0, 1]$ the fault's position within a schedule cycle, $p = 3$ the number of probes, and $\delta = 5$ seconds their spacing. An idealised model is

$$D_{\text{nominal}}(S, \phi) = S(1 - \phi) + (p - 1)\delta. \tag{7.1}$$

This excludes probe duration, schedule jitter, invocation latency, and subsequent workflow work. It is a planning model rather than a prediction or a bound on actual recovery. With uniformly distributed fault phase, its expected value is $S/2 + 10$: 40, 70, and 160 seconds for the three schedules.

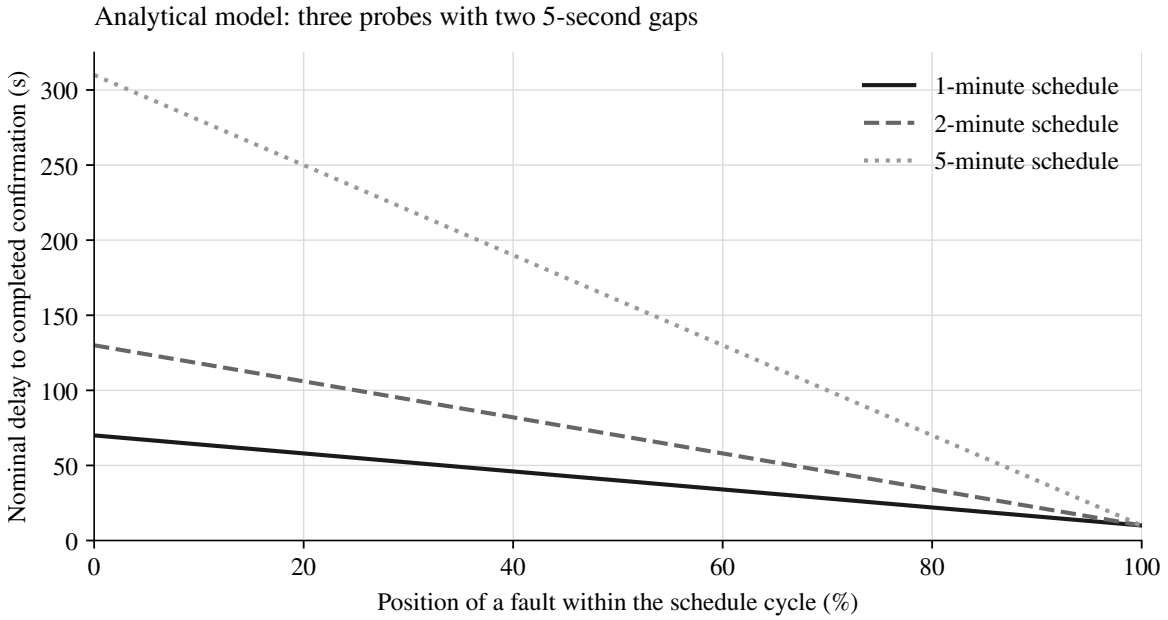


Figure 7.1: Analytical confirmation-delay scenarios, not measured performance. Faults just after a scheduled check wait almost a full cycle before confirmation can begin.

For h hours with monitoring enabled, the nominal scheduled activity is

$$N_{\text{scheduled}} = \frac{3600h}{S}. \tag{7.2}$$

At the implemented one-minute rate, a two-hour rehearsal produces 120 nominal monitor

triggers and continuous daily operation produces 1,440. These counts exclude retries, manual invocations, and recovery workflow calls. They cannot be multiplied by a single unit price to obtain the entire cloud bill.

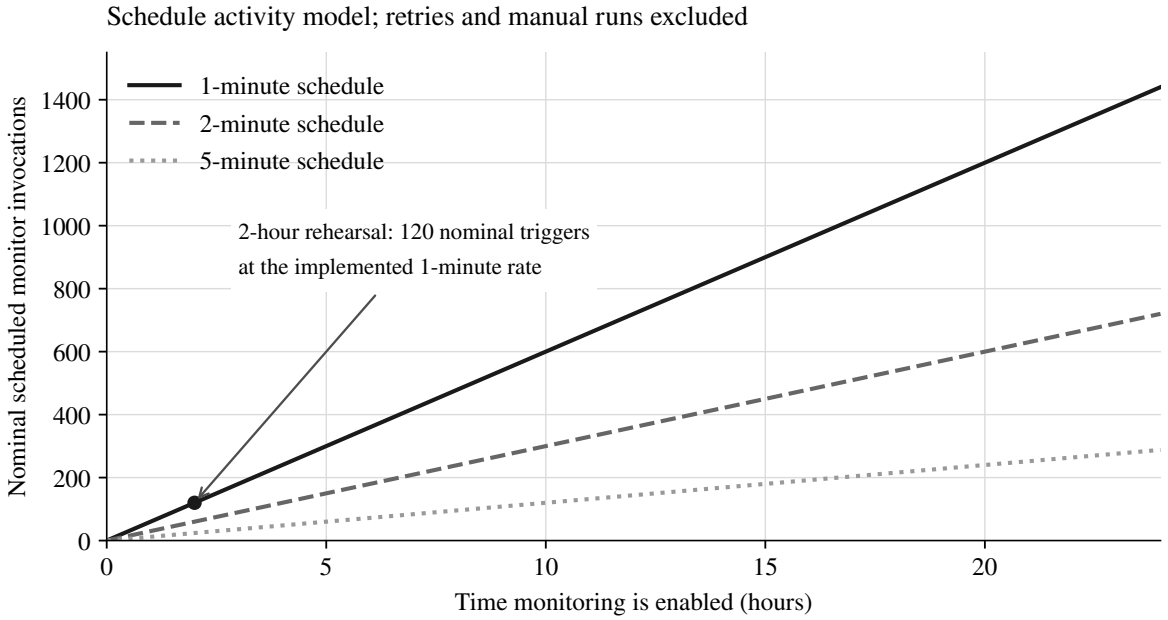


Figure 7.2: Analytical monitoring activity against enabled hours. A slower schedule reduces recurring calls while increasing the phase-dependent wait shown in Figure 7.1.

The appropriate policy depends on the acceptable interruption and the cost of a false promotion. For this small demonstration, three probes suppress a short-lived fault and avoid reacting to a single failed request. Lower latency would require changing and validating the detection path, not simply claiming that public health recovery is instantaneous. Repeated trials with randomised fault phase should precede any production RTO claim.

7.6 Cost Model

The major cost drivers are the two EC2 instances, their public IPv4 addresses while assigned, retained EBS volumes, DynamoDB operations and storage, Lambda invocations, API Gateway requests, workflow transitions, S3 objects, CloudFront delivery, and logs. Gateway VPC endpoints for S3 and DynamoDB have no additional endpoint charge [3]. The design avoids fixed NAT gateway and load-balancer resources.

A planning model separates active runtime from retained storage:

$$C_{\text{period}} = 2h(P_{\text{EC2}} + P_{\text{IPv4}}) + C_{\text{EBS}} + C_{\text{requests}} + C_{\text{storage}} + C_{\text{delivery/logs}}, \tag{7.3}$$

where h is the number of hours that both application instances are running. Rates depend on the selected Region and current pricing. This expression identifies cost drivers; it is not a fixed quotation or a guarantee that credits cover the deployment. Current

EC2, VPC, and EBS rates should be checked for the actual rehearsal period [6, 7, 8].

The HTTP API stage is configured for a rate of three requests per second and burst capacity of six. No reserved Lambda concurrency is allocated. These values bound the demonstration and must not be treated as a capacity plan for a large cohort. Pause/resume, limited log retention, local rehearsal, and deterministic builds support efficient use of the available resources.

7.7 Alignment with Cloud Architecture Principles

The AWS Well-Architected Framework describes six pillars [5]. Table 7.2 maps the implementation to those themes without claiming a formal framework review.

Table 7.2: Relationship to the Well-Architected pillars

Pillar	**Relevant project decision**
Operational excellence	Observable events, documented runbook, reproducible local checks
Security	Restricted ingress, service roles, separate operational token
Reliability	Standby verification, atomic fencing, receipt replay, outbox recovery
Performance efficiency	Bounded functions, timeouts, and explicit small-demo traffic limits
Cost optimisation	Local-first rehearsal, pause controls, absence of fixed NAT/load-balancer resources
Sustainability	Stop idle compute and retain only necessary demonstration artifacts

Chapter 8

Limitations, Future Work, and Conclusion

8.1 Current Limitations

8.1.1 Shared Data and Control Dependencies

Both application instances depend on the same regional DynamoDB service. The router, API Gateway, Lambda, and workflow services are also part of the operating boundary. A complete Region outage or shared database failure cannot be repaired simply by changing the active application instance. No cross-Region replication or recovery path is part of the supported project.

8.1.2 Timing and Capacity

The monitoring schedule, confirmation probes, service invocation overhead, and client polling affect recovery time. The recorded values come from a single rehearsal and do not establish a production latency distribution. The API traffic limits and small instances are suitable for the bounded demonstration, while high-concurrency assessment traffic remains unvalidated.

A backend failure does not automatically extend a student's allowed time. The deadline calculation uses a persisted start time and mutable assessment duration, so administrator policy changes can affect an active attempt. Deadline validation also precedes the final storage call. A production assessment service should snapshot the effective policy and define whether acceptance is measured at request arrival, validation, or database commit. Exact receipt replay resolves an earlier commit; it does not authorise a new late answer.

8.1.3 Deployment and Feature Coverage

Local mode uses shared memory in one process and loses local state when the whole launcher restarts. It cannot demonstrate infrastructure isolation. Cloud mode uses separate instances and persistent regional storage, but the observed instance-stop experiment is not a complete Availability Zone outage test.

The broader product includes classroom collaboration and boards. The cloud recovery stack exposes the bounded HTTP assessment path and does not demonstrate WebSocket/board failover. Screenshots can also show a different frontend revision from a recorded cloud experiment; the request and data records are the authority for the specific measured replay outcome.

8.2 Future Work

Useful extensions should preserve the existing submission and writer contracts:

- Measure repeated recovery trials across realistic traffic levels and report distributions with clearly defined timing boundaries.
- Add production observability, alerting, tracing, and a documented response process for shared-service failures.
- Exercise backup and restore, set explicit retention rules, and validate data handling before using real examination records.
- Freeze effective attempt policy and test delayed writes across the deadline boundary, including concurrent changes to duration and availability.
- Strengthen service-to-service transport protection and subject authentication, authorisation, and abuse controls to independent security testing.
- Evaluate a production routing and capacity design while retaining transaction-level writer safety and durable request identity.
- Define and test a separate recovery contract for collaborative boards and WebSockets if those features enter the cloud assessment scope.

8.3 Conclusion

Continuum demonstrates an application-centred approach to cloud recovery for timed assessments. It connects infrastructure failover with the correctness of the operation a student needs to complete. Readiness confirmation governs when recovery is justified, an incident lease serialises the decision, a monotonically increasing epoch assigns writer authority, and a database condition applies that authority at commit time.

Durable request receipts address the uncertainty created by a response lost after a successful commit. The transactional outbox permits pending audit work to complete without duplicating the logical outcome. Persisted attempt identity and start time provide continuity across backend changes under unchanged assessment metadata.

The Sydney rehearsal combined an actual EC2 stop, promotion, public verification, exact receipt replay, and independent record inspection. Backend tests complement these observations. Within the stated dependency boundary, Continuum demonstrates recovery of both an accessible service and a verifiable submission outcome.

References

[1] Amazon Web Services, "Amazon DynamoDB Transactions: How it works," *Amazon DynamoDB Developer Guide*. https://docs.aws.amazon.com/amazondynamodb/latest/developerguide/transaction-apis.html. Accessed 23 October 2026.

[2] Amazon Web Services, "Choosing workflow type in Step Functions," *AWS Step Functions Developer Guide*. https://docs.aws.amazon.com/step-functions/latest/dg/choosing-workflow-type.html. Accessed 23 October 2026.

[3] Amazon Web Services, "Gateway endpoints," *Amazon Virtual Private Cloud User Guide*. https://docs.aws.amazon.com/vpc/latest/privatelink/gateway-endpoints.html. Accessed 23 October 2026.

[4] Amazon Web Services, "Enable internet access for VPC-connected Lambda functions," *AWS Lambda Developer Guide*. https://docs.aws.amazon.com/lambda/latest/dg/configuration-vpc-internet.html. Accessed 23 October 2026.

[5] Amazon Web Services, "The pillars of the framework," *AWS Well-Architected Framework*. https://docs.aws.amazon.com/wellarchitected/latest/framework/the-pillars-of-the-framework.html. Accessed 23 October 2026.

[6] Amazon Web Services, "EC2 On-Demand Instance Pricing." https://aws.amazon.com/ec2/pricing/on-demand/. Accessed 23 October 2026.

[7] Amazon Web Services, "Amazon VPC Pricing." https://aws.amazon.com/vpc/pricing/. Accessed 23 October 2026.

[8] Amazon Web Services, "Amazon EBS Pricing." https://aws.amazon.com/ebs/pricing/. Accessed 23 October 2026.

[9] Continuum source repository, current implementation: internal/continuum, internal/clusterstate, internal/exams, internal/store, internal/httpapi, infra/demo, and statemachine/demo-failover.asl.json. Source inspected 23 October 2026.

[10] Continuum project documentation, *Implementation and review evidence*, docs/implementation-review.md; *Demonstration runbook*, docs/demo-runbook.md; and *Product quality-of-life features*, docs/qol-features.md.

[11] Continuum, recorded AWS rehearsal, 10 October 2026. Repository artifact: docs/evidence/cloud-rehearsal.json.

[12] Continuum, independent AWS record and workflow inspection, 10 October 2026. Repository artifact: docs/evidence/cloud-records.json.

[13] Continuum, live stale-writer registration check, 10 October 2026. Repository artifact: docs/evidence/cloud-stale-writer.json.

[14] Continuum, local application capture, 23 October 2026. Synthetic BCSE355L classroom and assessment; original screens in docs/report/screenshots/, captured fault state and events in docs/report/data/local-capture-summary.json. Cloud graph measurements and analytical model parameters are included in docs/report/data/.

[15] Vellore Institute of Technology, official VIT logo asset. https://vit.ac.in/wp-content/uploads/2023/06/blue_logo-copy-scaled.webp. Used on the cover page; accessed 23 October 2026.

Appendix A

Reproducibility Guide

A.1 Local Execution

The local launcher starts the frontend, public router, A/B HTTP servers, and shared memory store. Separate browser sessions distinguish the teacher/operator and learner. Credentials are read privately from the ignored runtime file.

```
cd web
bun install --frozen-lockfile
cd ..
make demo-local
```

The default frontend address is `http://localhost:3000`; the cockpit is at `/ops`. Local mode labels its shared-process topology. A full launcher restart resets its in-memory data.

A.2 Backend Verification Commands

The six package suites in Table 6.2 were rerun using:

```
go test -p 2 -race -timeout 5m \
  ./internal/continuum ./internal/store \
  ./cmd/demo-reconciler

go test -p 2 -race -timeout 5m \
  ./integration ./internal/httpapi ./internal/exams
```

The repository also supplies broader Go, frontend, build, Terraform, and browser checks. Their scope should be selected according to the part of the system changed.

A.3 Controlled Recovery Demonstration

1. Verify that both backends are ready, record the active backend and epoch, and start a fresh synthetic learner attempt.

2. Stop the active local HTTP server or, in an enabled AWS rehearsal, the active EC2 instance. Observe confirmation, promotion, and public verification.

3. Keep assessment policy unchanged and confirm that the learner's existing attempt start time continues to govern timing. Submit within the configured rules and retain

the receipt.

4. Using a fresh suitable attempt, inject a post-commit response loss and retry the same request and answer snapshot. Compare its request ID and commit time.

5. Repeat reconciliation and inspect the canonical submission, receipt, processed outbox, and audit result. An unchanged logical outcome is required.

6. Restore a healthy baseline, exercise the brief-fault and unsafe-standby cases, and verify that inappropriate switches are suppressed.

A.4 Evidence and Operating Boundaries

The preserved cloud artifacts are the rehearsal JSON, independent record/workflow inspection, and stale-registration check [11, 12, 13]. Their results apply to the recorded synthetic scenarios. Assessment screenshots illustrate interface states and are not substitutes for request comparisons or database inspection.

Cloud execution requires a verified project plan, selected Region, current spend status, and healthy instances. Monitoring and compute should be paused after a bounded rehearsal. Resource destruction is separate from pause because it removes retained data.

A.5 Rebuilding the Report and Graphs

The LaTeX source package contains all included screenshots, vector diagrams, chart PDFs, whitelisted measurements, and chart-generation code. It compiles without a running application or an AWS connection:

```
mkdir -p build
tectonic --keep-logs --outdir build main.tex
```

The charts are already included as vector PDFs. To regenerate them, install Python's Matplotlib and NumPy packages, then run:

```
python figures/build_figures.py
```

The script reads `data/cloud-measurements.json`, exports the aligned event CSV, and writes five PDF/PNG chart pairs. Recorded measurements and analytical assumptions occupy separate data files. Screenshot crops in LaTeX change framing only; they do not alter the interface state or numerical results.